
BỘ TÀI LIỆU KỸ THUẬT · TÀI LIỆU 02

LAAM

Kiến trúc hệ thống

Các thành phần, luồng xử lý request, engine agent và engine workflow, tầng mô hình và tầng connector, mô hình dữ liệu và bề mặt tích hợp của một trợ lý AI cho công việc, tự host và đa người dùng.



PHIÊN BẢN
1.0

NGÀY
2026-09-24

PHIÊN BẢN MÃ NGUỒN
LAAM@7eb0724

Mục lục

1 Tổng quan và nguyên tắc thiết kế

2 Bối cảnh hệ thống

3 Kiến trúc thành phần

4 Engine chat và agent

Vòng đời một lượt hỏi · thứ tự ưu tiên prompt và persona · vòng lặp tool · giao thức streaming · mức độ suy luận · quản lý ngữ cảnh

5 Tầng nhà cung cấp mô hình

6 Tool và connector

Tool nội bộ · connector tích hợp sẵn · MCP client · xử lý kết quả

7 Các guard độ tin cậy

8 Engine workflow

Mô hình graph · chế độ thực thi · độ bền vững và ghi idempotent · trigger và bộ lập lịch

9 Soạn workflow bằng AI

10 Pipeline giọng nói

11 Mô hình dữ liệu

12 Bề mặt API

13 Bộ công cụ đánh giá

A Phụ lục — Tham chiếu endpoint API

Thuật ngữ trạng thái dùng trong toàn bộ tài liệu: **LIVE** đã hiện thực, đã tích hợp và đã chạy với dữ liệu thật · **BUILT** đã hiện thực và đã tích hợp, chưa chạy với dữ liệu production · **PLANNED** nằm trong lộ trình, tắt mặc định qua cờ cấu hình, hoặc mới chỉ có interface. Trạng thái triển khai: **giai đoạn tiền production (chưa triển khai production)** — hiện LAAM chạy dưới dạng stack Docker Compose tự host, phục vụ phát triển, sử dụng nội bộ và trình diễn.

1 Tổng quan và nguyên tắc thiết kế

LAAM là trợ lý AI cho công việc, tự host và đa người dùng. Người dùng đặt câu hỏi nghiệp vụ qua chat hoặc giọng nói; LAAM trả lời dựa trên dữ liệu thực bằng cách gọi tool trên các hệ thống đã kết nối. Trong chat, mọi thao tác ghi lên một hệ thống đã kết nối đều chờ người dùng xác nhận một bản xem trước do mã dựng; workflow chạy theo lịch chỉ được ghi qua các tool được đánh dấu an toàn cho workflow và chỉ tới những người nhận nằm trong whitelist. Cùng tầng tool này vận hành các graph workflow bền vững, có thể lập lịch, do AI soạn và chạy thử thay cho người dùng.

LAAM là một ứng dụng Next.js 16 duy nhất (React 19, TypeScript strict) dùng PostgreSQL 16 làm cơ sở dữ liệu. Ứng dụng cung cấp bảy bề mặt sản phẩm:

Bề mặt	Chức năng	Trạng thái
Chat /chat	Trợ lý LLM đa nhà cung cấp, gọi tool qua connector và MCP server, thẻ xác nhận thao tác ghi, đính kèm tệp (PDF, DOCX, hình ảnh, OCR), hiển thị phong phú (bảng, biểu đồ, bản đồ) và workspace riêng cho từng người dùng với persona dùng chung.	LIVE
Giọng nói /constellation	Trung tâm điều khiển bằng giọng nói toàn màn hình, không cần chạm tay: nhận dạng giọng nói trên trình duyệt, ngắt lời dựa trên phát hiện giọng nói, chuyển văn bản thành giọng nói bằng mạng nơ-ron dạng streaming từ một dịch vụ tự host.	LIVE
Workflow /workflows	Tự động hoá dạng graph bền vững (các node agent, connector, MCP, điều kiện, for-each; tùy chọn thực thi song song), lịch cron, chạy thử (dry-run), và soạn bằng AI có chạy thử các graph mà AI đề xuất. Engine workflow ở trạng thái LIVE; bộ lập lịch cron và tính năng soạn bằng AI ở trạng thái BUILT.	LIVE / BUILT
Connector /connectors	12 connector tích hợp sẵn (11 tích hợp và 1 bản demo offline) cùng số lượng tùy ý MCP server từ xa cho từng người dùng, với thông tin xác thực mã hoá theo từng người dùng và OAuth khi nhà cung cấp hỗ trợ.	LIVE / BUILT
Tìm kiếm /search	Tìm kiếm toàn cục trên các phiên được giám sát, các cuộc hội thoại của người gọi (bao gồm nội dung tin nhắn) và workflow, dùng index trigram.	BUILT
Quyền truy cập /settings/*	Kiểm soát truy cập theo vai trò (owner / admin / member / viewer), thu hồi quyền khi nghỉ việc, token truy cập API và MCP cá nhân, khoá do admin cấp, agent tùy chỉnh theo từng người dùng.	LIVE
Giám sát /monitoring	Xem theo thời gian thực các phiên của AI coding agent (được collector nạp từ các tệp transcript) và các lượt chat, lượt chạy workflow của chính LAAM, bao gồm mô hình, tool, token và chi phí.	LIVE

LAAM cũng là một MCP server **BUILT**: agent bên ngoài có token truy cập có thể gọi chín tool chỉ đọc qua `POST /api/mcp`.

26 ROUTE TRANG	88 HANDLER PHƯƠNG THỨC API (64 TỆP ROUTE)	22 BẢNG POSTGRES · 19 MIGRATION	363 TỆP TEST (VITEST)
--------------------------	---	---	---------------------------------

12

CONNECTOR TÍCH HỢP
SẴN · 46 TOOL

12

TOOL NỘI BỘ CỦA
AGENT

5

NHÀ CUNG CẤP LLM
ĐƯỢC HỖ TRỢ

3

NGÔN NGỮ GIAO DIỆN
(VI · EN · ZH)

Mọi số liệu được đếm từ cây mã nguồn tại commit `7eb0724`. 12 connector tích hợp sẵn gồm 11 tích hợp và 1 bản demo offline; 14 trong 46 tool của chúng là tool ghi. Bộ Vitest gồm 3,573 test case trong 363 tệp (3,567 pass khi chạy lại trên phiên bản mã nguồn này, ngày 2026-09-24).

• Nguyên tắc thiết kế

Năm nguyên tắc lặp lại xuyên suốt mã nguồn và giải thích phần lớn kiến trúc được mô tả ở các chương sau. Mỗi nguyên tắc được hỗ trợ bằng mã hoặc test, không chỉ dựa vào quy ước.

Nguyên tắc	Cách thực thi
Chỉ dùng mô hình cho các quyết định cần phán đoán	Định tuyến, thử lại, trích xuất, tính toán số học, dựng bảng và thực thi thao tác ghi đều do mã đảm nhiệm. Mô hình quyết định gọi <i>tool</i> nào và diễn đạt câu trả lời ra sao. Ví dụ: bảng kết quả được mã dựng từ các dòng dữ liệu của tool và stream thành một frame riêng, nên mô hình không bao giờ phải gõ lại các con số.
Con người xác nhận thao tác ghi trong chat	Một tool chỉ được coi là tool đọc khi được khai báo chỉ đọc bởi một connector tích hợp sẵn, hoặc bởi một MCP server mà người dùng đã đánh dấu tin cậy; mọi tool khác được coi là tool ghi. Trong chat, mọi thao tác ghi lên một hệ thống đã kết nối đều tạm dừng lượt hỏi và chờ người dùng xác nhận một bản xem trước do mã dựng; lần gọi đã xác nhận chạy bằng mã với các tham số đã niêm phong, và nonce của nó được đối chiếu với nhật ký audit. Workflow chạy theo lịch hoặc không có người giám sát chỉ được thực thi thao tác ghi với các tool được đánh dấu an toàn cho workflow và chỉ tới những người nhận nằm trong whitelist; mọi thao tác ghi khác bị chặn (fail closed).
Báo lỗi rõ ràng	Không tự động chuyển sang nhà cung cấp khác một cách âm thầm, không trả lời rỗng, không báo "thành công" khi chưa có thao tác ghi nào được thực thi. Mỗi đường lỗi tạo ra một thông báo có mã, bằng ba ngôn ngữ, hoặc một bước thất bại nêu rõ nguyên nhân.
Không phụ thuộc nhà cung cấp mô hình	Năm nhà cung cấp nằm sau một hợp đồng adapter nội bộ duy nhất. Người dùng chat tự chọn mô hình; các tác vụ nền xác định mô hình qua một bộ định tuyến tất định. Năng lực (tool, thị giác, đầu ra có cấu trúc) được kiểm tra theo từng nhà cung cấp.
Không phụ thuộc connector	Hệ thống bên ngoài được cắm vào qua interface <code>Connector</code> chung hoặc dưới dạng MCP server do người dùng thêm, và tầng tích hợp là tầng chung. Hành vi riêng của từng connector (ví dụ các cặp đi sâu từ danh sách đến chi tiết) được cung cấp qua cấu hình. Một vài chuỗi prompt gửi cho mô hình vẫn nhắc đến tool của một knowledge graph server cụ thể; việc dọn dẹp các chuỗi này đang được theo dõi.

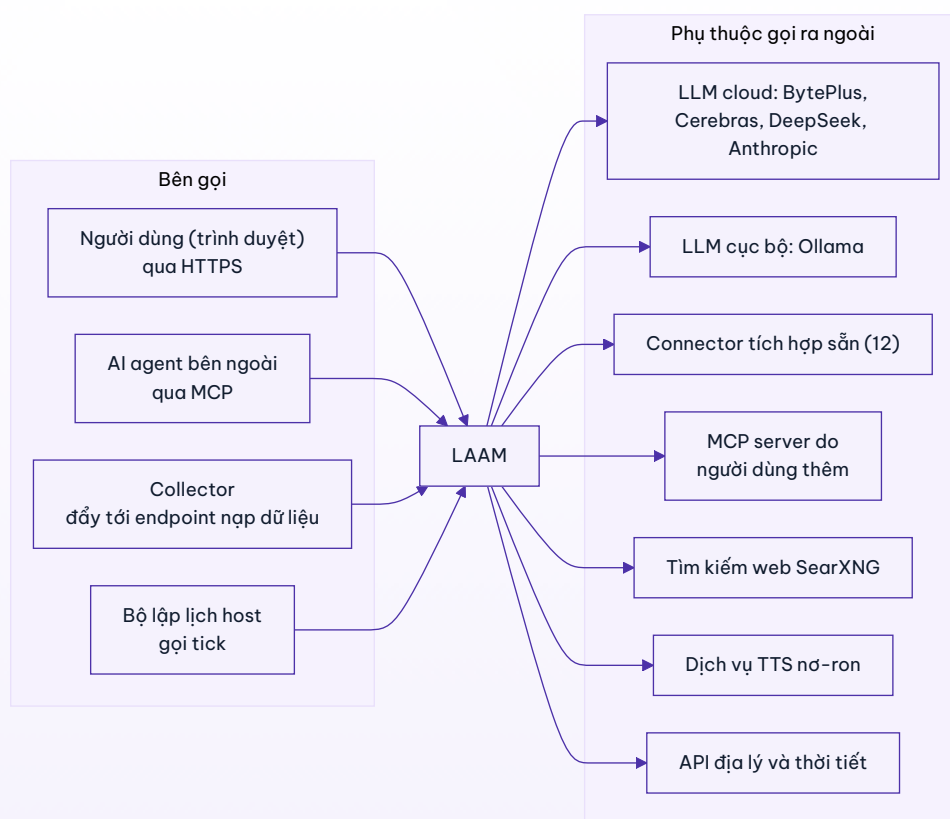
Agent không thực thi mã

Một test trong bộ unit test sẽ thất bại nếu việc tạo tiến trình, `eval`, `new Function` hoặc module `vm` xuất hiện ở bất kỳ đâu trong engine agent hoặc engine workflow; hiện chưa có CI, nên bộ test được chạy thủ công. Các tool nội bộ và các loại node workflow được cố định trong mã, và test này cũng kiểm tra tên và số lượng tool. Trớ lý chỉ có thể hành động thông qua các tool đã khai báo.

2 Bối cảnh hệ thống

LAAM chạy dưới dạng một container ứng dụng trên host do khách hàng kiểm soát, bên cạnh cơ sở dữ liệu và hai dịch vụ sidecar tùy chọn. Mọi thông tin trợ lý biết về thế giới bên ngoài đều đến qua một trong bốn kênh: nhà cung cấp LLM, connector (tích hợp sẵn hoặc MCP), tìm kiếm web, và các dịch vụ địa lý, thời tiết công khai. Người dùng truy cập từ trình duyệt trên máy tính hoặc thiết bị di động.

Tác nhân bên ngoài	Tương tác	Thông tin xác thực
Người dùng	Lướt chat, lướt giọng nói, soạn và chạy workflow, quản lý connector, quản trị.	Email và mật khẩu (bcrypt), phiên JWT
AI agent bên ngoài	Gọi chín tool giám sát chỉ đọc của LAAM qua MCP; bản thân mỗi lần gọi cũng được ghi lại thành một phiên được giám sát.	Token truy cập cá nhân (loại <code>api</code> hoặc <code>mcp</code>)
Nhà cung cấp LLM	Chat completion, gọi tool, đầu ra có cấu trúc. Khóa chỉ nằm phía server; không bao giờ gửi xuống trình duyệt.	API key của đơn vị vận hành (biến môi trường)
Connector tích hợp sẵn	Đọc và ghi qua REST API của từng nhà cung cấp.	Token hoặc quyền OAuth theo từng người dùng, mã hoá khi lưu trữ
MCP server	Khám phá tool (<code>tools/list</code>), gọi tool, chỉ dẫn của server.	Bearer token và header theo từng người dùng, mã hoá khi lưu trữ
Collector	Đẩy metadata phiên coding agent đã được phân tích, kèm các đoạn văn bản ngắn đã được che theo mẫu (không bao giờ gửi toàn bộ transcript), từ máy của lập trình viên.	Token truy cập của collector
Bộ lập lịch của host	Gọi endpoint tick của workflow mỗi phút một lần.	Secret dùng chung (so sánh thời gian hằng)



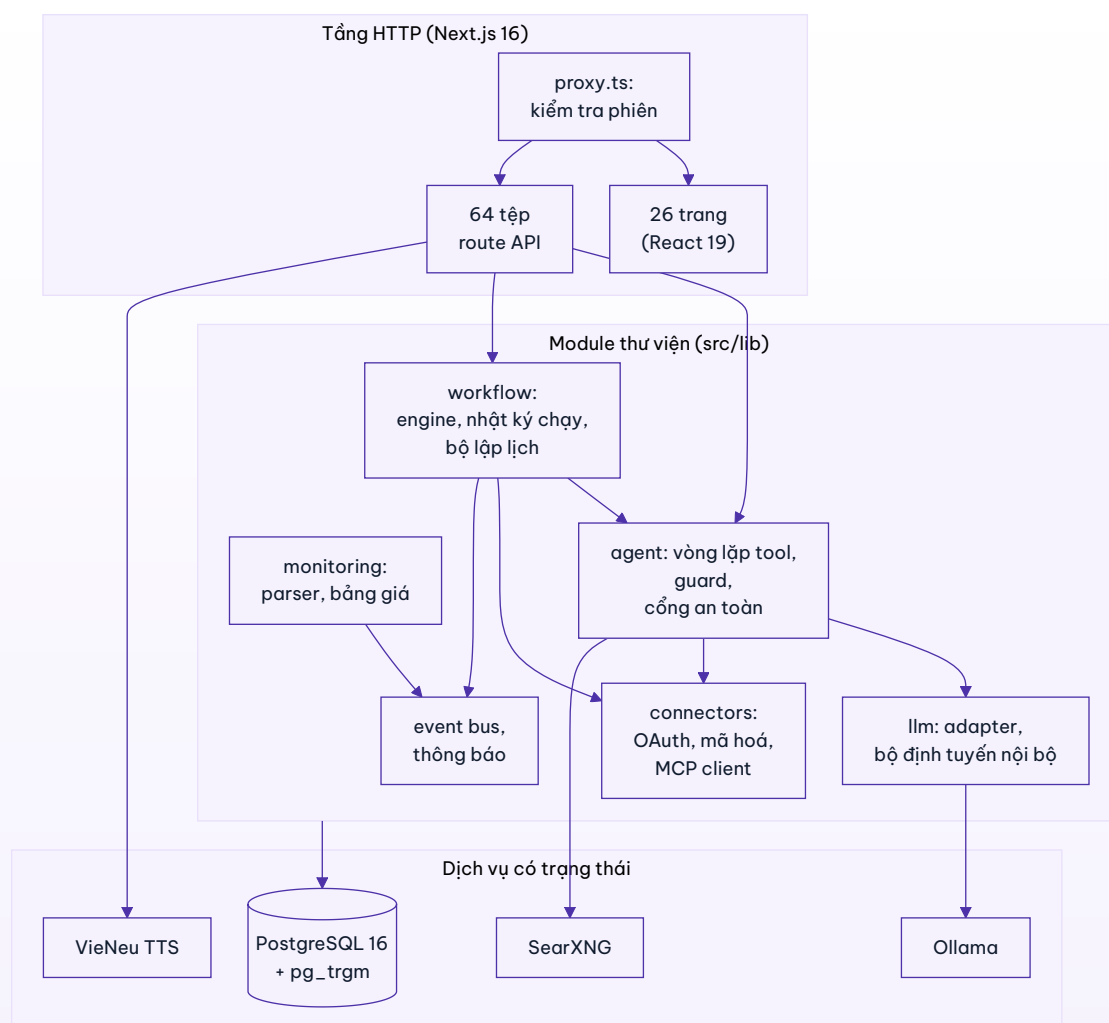
Hình 1. Bối cảnh hệ thống. LAAM là thành phần duy nhất người dùng giao tiếp trực tiếp; mọi thông tin xác thực của mô hình và connector đều nằm phía server.

• Mô hình triển khai

Triển khai tham chiếu là một stack Docker Compose trên một host duy nhất có GPU: container ứng dụng LAAM (image `node:24-alpine` dựng nhiều giai đoạn, đầu ra standalone của Next.js, chạy bằng người dùng không phải root, có health check; kèm sẵn các binary OCR và PDF), dịch vụ TTS, SearXNG và một giao diện quản trị cơ sở dữ liệu. PostgreSQL 16 là một instance bên ngoài, dùng chung với các dịch vụ khác trên mạng của host. Ollama và một agent nhỏ thu thập số liệu host chạy trực tiếp trên host để truy cập được GPU. Lưu lượng vào đi qua một mạng VPN dạng lưới riêng, và có thể tùy chọn công khai ra internet qua tính năng tunnel của chính VPN đó. Hạ tầng, bảo mật và vận hành được trình bày trong tài liệu 03 của bộ tài liệu này.

3 Kiến trúc thành phần

Bên trong ứng dụng, một tầng HTTP mỏng (các trang App Router và route handler của API) nằm trên một tập module thư viện trong `src/lib`. Route handler đảm nhiệm xác thực, kiểm tra request và streaming; các module thư viện đảm nhiệm toàn bộ hành vi và được unit test mà không cần server.



Hình 2. Kiến trúc thành phần. Nhà cung cấp LLM cloud và API của connector (không thể hiện trên hình) được gọi từ các module llm và connectors ; các module auth và mcp-server được lược bỏ cho dễ đọc.

Thành phần	Trách nhiệm	Vị trí
Bảo vệ route	Kiểm tra phiên (tương thích edge) trên mọi đường dẫn không phải tệp tĩnh; một danh sách công khai ngắn (trang giới thiệu, đăng nhập, đăng ký, và ba endpoint xác thực bằng token hoặc secret: nạp dữ liệu, MCP, tick của workflow).	src/proxy.ts
Route chat	Một lượt chat: kiểm tra đầu vào, nạp hội thoại, giới hạn độ dài lịch sử, tập tool, prompt, rẽ nhánh theo nhà cung cấp, các frame streaming, tạm dừng thao tác ghi và xác nhận.	app/api/chat/route.ts
Engine agent	Vòng lặp tool (<code>runToolRounds</code>), cổng an toàn, guard và nudge, bảng dữ liệu và bản tóm lược (digest) kết quả, loại bỏ ngữ cảnh, tóm tắt.	lib/agent
Tầng nhà cung cấp	Adapter cho Ollama, BytePlus, Cerebras, DeepSeek và Anthropic; bộ định tuyến mô hình nội bộ; các chiến lược đầu ra có cấu trúc.	lib/llm
Tầng connector	12 connector tích hợp sẵn sau một interface duy nhất; luồng OAuth và làm mới token dưới advisory lock của Postgres; kho thông tin xác thực AES-256-GCM; MCP client chung có khám phá tool, đặt namespace và guard SSRF.	lib/connectors

Thành phần	Trách nhiệm	Vị trí
Engine workflow	Bộ duyệt tuần tự và bộ lập lịch DAG song song, executor của node, nhật ký chạy, sổ cái idempotency, khôi phục sau sự cố, bộ lập lịch cron, và pipeline soạn bằng AI.	lib/workflow
Xác thực	Đăng nhập bằng thông tin xác thực qua Auth.js v5 với phiên JWT được làm mới từ cơ sở dữ liệu; các helper RBAC; khoá đăng nhập sau nhiều lần sai và giới hạn tốc độ (rate limiting) đăng ký.	lib/auth , src/auth.ts
Giám sát	Parser transcript (dùng chung với collector), bảng giá, phát hiện phiên bị kẹt, read model thống nhất cho các lượt chạy của agent, chat và workflow.	lib/monitoring
Event bus	Publish/subscribe trong tiến trình, gắn trên đối tượng global; phân phối tới endpoint SSE (/api/events) cho phiên, bước workflow và thông báo theo từng người dùng.	lib/events-bus.ts
Hook khởi động	Khi khởi động, đánh dấu các lượt chạy workflow bị bỏ dở ở trạng thái <code>running</code> thành <code>resumable</code> để bộ lập lịch tiếp tục chúng.	src/instrumentation.ts
Collector / agent host	Hai script Node không phụ thuộc thư viện ngoài: collector đẩy metadata phiên đã phân tích từ máy của lập trình viên; agent host cung cấp số liệu CPU, RAM và GPU cho bề mặt giọng nói.	collector/ , host-agent/

Công nghệ sử dụng

Tầng	Công nghệ
Runtime và framework	Node.js 24, Next.js 16.2 App Router (đầu ra standalone), React 19.2, TypeScript 6 (strict), Tailwind CSS 4
Xác thực	Auth.js v5 beta (<code>next-auth 5.0.0-beta.31</code>) với adapter Drizzle, băm mật khẩu bằng bcrypt, chiến lược phiên JWT
Dữ liệu	PostgreSQL 16 với <code>pg_trgm</code> ; Drizzle ORM và migration bằng Drizzle Kit; driver <code>node-postgres</code> ; kiểm tra dữ liệu bằng Zod 4
LLM	Anthropic SDK cho Claude; HTTP tương thích OpenAI (không dùng SDK) cho BytePlus ModelArk, Cerebras và DeepSeek; Ollama HTTP API
MCP	Client: <code>@modelcontextprotocol/sdk</code> chính thức (Streamable HTTP, dự phòng bằng SSE). Server: JSON-RPC viết tay qua Streamable HTTP
Thư viện giao diện	React Flow (trình soạn workflow, graph phiên), Recharts, Leaflet với OpenStreetMap, three.js / react-three-fiber (hiệu ứng hình ảnh cho giọng nói), react-markdown có làm sạch nội dung
Giọng nói	Web Speech API (chuyển giọng nói thành văn bản), Silero VAD trên trình duyệt, VieNeu TTS tự host (Python, FastAPI)
Tài liệu	Tesseract OCR (tiếng Việt, tiếng Anh, tiếng Trung giản thể), trích xuất PDF bằng Poppler, trích xuất văn bản DOCX
Kiểm thử	Vitest, Testing Library, jsdom; một cấu hình đánh giá riêng chỉ chạy trên host, gọi mô hình thật

Thiết kế một tiến trình

Event bus, bộ giới hạn tốc độ và logic tiếp tục workflow giả định chỉ có một tiến trình ứng dụng. Điều này giữ cho stack đơn giản với các triển khai tự host; khi cần mở rộng theo chiều ngang, bước đầu tiên là chuyển bus và bộ giới hạn sang một kho dùng chung (ví dụ Redis).

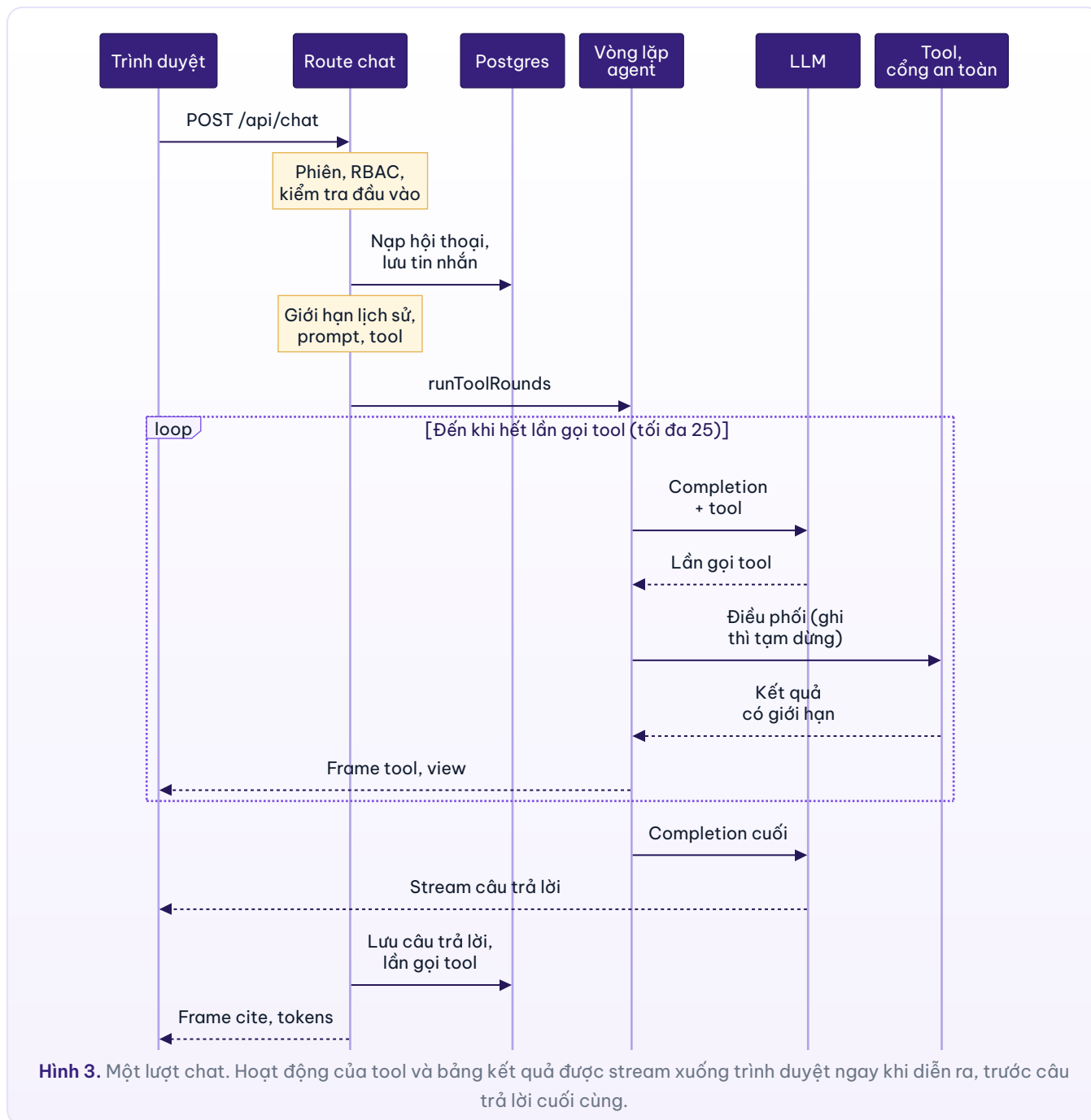
4 Engine chat và agent

Một lượt chat là một request `POST /api/chat`. Route xác thực người gọi, tập hợp ngữ cảnh, chạy vòng lặp tool với mô hình đã chọn, và stream câu trả lời về trong khi lưu lại mọi tin nhắn và lần gọi tool. Cùng engine này phục vụ bề mặt giọng nói, chỉ khác ở hướng dẫn đầu ra.

4.1 Vòng đời một lượt hỏi

Route thực hiện các bước sau theo thứ tự:

- 1. Phân quyền.** Yêu cầu có phiên, sau đó yêu cầu vai trò được phép thay đổi dữ liệu (viewer chỉ được đọc). Trước mọi thao tác I/O, id mô hình Claude được đối chiếu với whitelist (nếu không khớp thì trả về 400); id của BytePlus và Cerebras được nhận diện theo danh sách, và mọi id khác được chuyển tới runtime cục bộ.
- 2. Kiểm tra đầu vào.** Từ chối tin nhắn rỗng, loại bỏ byte NUL, giới hạn tối đa hai hình ảnh mỗi lượt và khoảng 2.8 MB (base64) mỗi hình.
- 3. Nạp ngữ cảnh.** Nạp hoặc tạo hội thoại. Hội thoại mới được gắn với workspace của nó sau khi kiểm tra quyền sở hữu; workspace không bao giờ thay đổi sau đó.
- 4. Giới hạn độ dài lịch sử.** Phát lại lịch sử trong một ngân sách ký tự tùy theo nhà cung cấp. Phần tràn được gộp vào một bản tóm tắt cuốn chiếu bởi mô hình nội bộ (chương 5) và lưu trên dòng dữ liệu của hội thoại.
- 5. Dựng tập tool.** Tool nội bộ, cộng với tool của các connector tích hợp sẵn mà người dùng đã kết nối, cộng với mọi tool từ các MCP server của người dùng, đặt namespace theo từng server (chương 6).
- 6. Điều phối trước một tool nhanh** (tùy chọn). Nếu người dùng chọn một tool cụ thể trên giao diện, mã sẽ chạy nó một cách tắt định trước vòng lặp mô hình, qua cùng cổng an toàn.
- 7. Tập hợp system prompt** (mục 4.2), sau đó thêm cảnh báo chủ động (phiên được giám sát bị kẹt hoặc tốn kém, khử trùng lặp theo từng hội thoại với thời gian chờ 6 giờ) dưới dạng một frame riêng, không bao giờ nằm trong văn bản do mô hình viết.
- 8. Chạy vòng lặp tool** (mục 4.3), sau đó là completion cuối (mục 4.6).
- 9. Hoàn tất.** Loại bỏ JSON gọi tool bị lọt vào câu trả lời, lưu tin nhắn trợ lý kèm số token, lưu mọi lần gọi tool và kết quả, phát các frame cuối.
- 10. Hoặc tạm dừng.** Nếu mô hình đề xuất một thao tác ghi, lượt hỏi kết thúc bằng một thẻ xác nhận (chương 7).



4.2 System prompt và thứ tự ưu tiên persona

System prompt được mã tập hợp từ sáu phần theo thứ tự cố định: persona cơ sở, ngày hôm nay, gợi ý ngôn ngữ (tiếng Việt, tiếng Anh hoặc tiếng Trung), khối tool (chỉ khi có tool), chỉ dẫn riêng của từng MCP server (lấy từ phản hồi `initialize` của server, có giới hạn độ dài), và một hướng dẫn đầu ra. Hướng dẫn cho văn bản quy định hợp đồng về khối bảng, biểu đồ và bản đồ; hướng dẫn cho giọng nói đặt văn phong nói, cấm đọc to các định danh và chỉ cho phép tối đa một bảng hoặc biểu đồ.

Khối tool chứa các quy tắc hành vi đã được đo lường, được diễn đạt theo cách chung: thao tác ghi phải đi qua tool và không bao giờ được báo thành công khi không có tool; "tìm lại" buộc phải gọi mới; số đếm lấy từ dữ liệu có cấu trúc; so sánh nhiều chỉ số được tách thành các lần gọi riêng; tool dữ liệu dạng ngôn ngữ tự nhiên nhận nguyên văn câu chữ của người dùng; SQL không bao giờ được đặt vào trường ngôn ngữ tự nhiên.

Ưu tiên	Nguồn	Tác động
1	Ghi đề system prompt tường minh	Thay thế toàn bộ prompt; các khối tool, ngày, ngôn ngữ và cảnh báo chủ động bị bỏ. Dùng cho người dùng nâng cao và các đường xử lý nội bộ.
2	Persona của workspace	Chỉ thay persona cơ sở; các khối tool, ngày và ngôn ngữ vẫn giữ. Được xác định phía server từ dòng dữ liệu của hội thoại, nên một lượt giọng nói tiếp tục ở nơi khác vẫn có cùng persona.
3	Agent tùy chỉnh do người dùng chọn	Chỉ thay persona cơ sở.
4	Mặc định của hệ thống	Persona trợ lý tích hợp sẵn.

Persona của workspace là một agent tùy chỉnh hoặc chỉ dẫn dạng văn bản tự do, không bao giờ là cả hai (API từ chối trường hợp có cả hai với mã 400). Agent tùy chỉnh đã bị xóa sẽ quay về mặc định và giao diện hiển thị nhãn cảnh báo.

4.3 Vòng lặp tool

`runToolRounds` (`lib/agent/orchestrator.ts`) chạy cho đến khi mô hình ngừng gọi tool. Từ phiên bản 2.5.0, cơ chế này đã thay thế giới hạn cố định bốn vòng, vốn cắt ngang mọi tác vụ cần hơn ba vòng tool (ví dụ tóm tắt mười email rồi gửi báo cáo).

Cơ chế	Hành vi
Chặn chạy vô hạn	Mặc định 25 vòng (có thể cấu hình, trần cứng 50). Vòng cuối bị buộc trả về văn bản.
Chặn lặp lại	Cùng một tool với tham số giống hệt ba lần được coi là bị kẹt (tám lần với các tool thăm dò trạng thái). Vòng lặp ngừng điều phối và yêu cầu mô hình trả lời với những gì đã có. "Bị kẹt" và "hết số vòng" được báo cáo riêng để đơn vị vận hành tinh chỉnh.
Loại bỏ ngữ cảnh	Khi hội thoại tích lũy vượt quá ngân sách ký tự (khoảng 37k ký tự với mô hình cục bộ, khoảng 400k với mô hình cloud), các kết quả tool cũ nhất được thay bằng một bản rút gọn, giữ lại ba kết quả gần nhất. Cách làm này theo mẫu xóa kết quả tool mà các nhà cung cấp mô hình lớn đang dùng.
Xử lý từng kết quả	Mỗi kết quả được bóc tách, kiểm tra rỗng, chuyển thành bảng dữ liệu nếu có dạng bảng, và được chú thích để mô hình không gõ lại một bảng đã hiển thị trên màn hình (chương 6).
Nudge một lần	Mỗi loại tối đa một lần mỗi lượt: đọc một trang sau khi tìm kiếm web trả về URL; hỏi lại khi mô hình trả lời từ trí nhớ ở vòng 0 dù có tool; ngừng dùng nhật ký audit của chính LAAM cho câu hỏi nghiệp vụ. Node agent trong workflow tắt nudge vì prompt của node chính là chỉ dẫn.

4.4 Giao thức streaming

Phản hồi chat là một stream `text/plain`, không phải Server-Sent Events. Văn bản trả lời thuần được xen kẽ với các frame điều khiển JSON, phân tách bằng ký tự record separator ASCII (U+001E), ký tự không bao giờ xuất hiện trong văn bản thông thường. Id của hội thoại được truyền trong một header của phản hồi. Ngược lại, các lượt chạy workflow phát sự kiện qua bus SSE tại `/api/events`.

Frame	Mục đích
<code>tool</code>	Một lần gọi tool bắt đầu hoặc trả về (vết trực tiếp trên giao diện).
<code>view</code>	Một bảng dữ liệu do mã dựng (bảng, tùy chọn gợi ý biểu đồ) từ kết quả của tool.
<code>cite</code>	Các nguồn được câu trả lời sử dụng.

Frame	Mục đích
pending_write	Một thao tác ghi đang chờ con người xác nhận: bản xem trước do mã dựng và token đã niêm phong.
proactive	Cảnh báo về các phiên được giám sát bị kẹt hoặc tốn kém.
tokens	Mức sử dụng token, chỉ gửi khi nhà cung cấp thực sự báo cáo (không bịa ra chi phí bằng 0).
hb	Heartbeat trong khi mô hình suy luận đang suy nghĩ, để các proxy không ngắt kết nối vì nhàn rỗi.

4.5 Mức độ suy luận theo giai đoạn

Với các mô hình suy luận trên BytePlus và Cerebras, LAAM đặt mức độ suy luận riêng cho **các vòng tool** và cho **câu trả lời cuối** (mỗi giai đoạn quay về một thiết lập dùng chung; chỉ các giá trị `low`, `medium` và `high` được chuyển tiếp). Điều này cho phép một bản triển khai đánh đổi giữa độ trễ và độ sâu ở đúng chỗ cần thiết. Cấu hình mẫu đi kèm đặt giai đoạn cuối ở mức `low` và cố ý để các vòng tool ở mặc định của nhà cung cấp: trong thử nghiệm, mức thấp khi chọn tool đã dẫn đến câu trả lời sai "không tìm thấy gì" cho một câu hỏi tra cứu dữ liệu. DeepSeek có thiết lập riêng, và chế độ "thinking" của nó bị tắt ở các vòng tool sau khi truy vết được một lần gọi kéo dài hơn 100 giây.

4.6 Câu trả lời cuối và quản lý ngữ cảnh

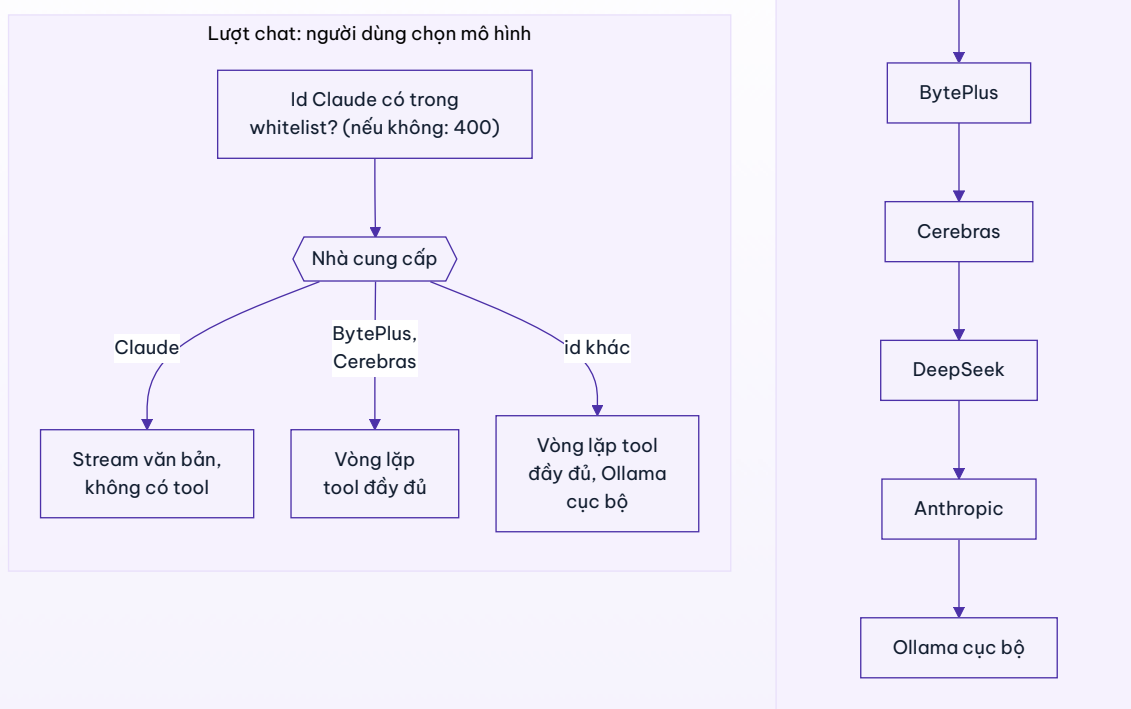
- **Bản tóm lược trước completion cuối.** Kết quả dạng bảng lớn đến với mô hình dưới dạng bản tóm lược (số dòng, các cột, tổng theo từng cột do mã tính trên toàn bộ các dòng, một vài dòng mẫu). Các dòng thô vẫn nằm trong `chat_tool_call`, nên khi nạp lại hội thoại, các bảng được dựng lại chính xác.
- **Ghim câu hỏi.** Sau một lượt có dùng tool trên nhánh cloud (BytePlus hoặc Cerebras), câu hỏi của người dùng được nhắc lại trước completion cuối. Cơ chế này được thêm sau một trường hợp đo được, khi câu trả lời trôi sang một chủ đề trước đó.
- **Khôi phục khi câu trả lời rỗng.** Nếu stream cuối của một mô hình cloud rỗng, engine thử lại đúng một lần với nudge "trả lời ngay từ những gì đã có"; nếu vẫn rỗng, engine gửi một thông báo có mã. Không bao giờ hiển thị bong bóng trống.
- **Tóm tắt cuốn chiếu.** LAAM không có kho bộ nhớ dài hạn riêng. Bộ nhớ hội thoại là bản tóm tắt cuốn chiếu trên dòng dữ liệu của hội thoại cộng với các lượt tool đã lưu. Mọi bộ nhớ dùng chung mà khách hàng muốn đều đi qua một MCP server đã kết nối như mọi tool khác.

5 Tầng nhà cung cấp mô hình

LAAM giao tiếp với năm nhà cung cấp mô hình qua một hợp đồng adapter duy nhất. Người dùng chọn mô hình chat trong bộ chọn; các tác vụ nền (tóm tắt, node agent của workflow, soạn bằng AI) không có bộ chọn và xác định mô hình qua một bộ định tuyến tất định.

Nhà cung cấp	Giao thức truyền	Bộ chọn chat	Vòng lặp tool	Thị giác	Ghi chú
Ollama (cục bộ)	Ollama HTTP API	Luôn có	Có	Có (≤ 2 hình)	Chi phí biên bằng 0; chạy trên GPU của host.
BytePlus ModelArk	HTTP tương thích OpenAI	Khi có khoá (chọn sẵn)	Có	Chưa kết nối	Tám mô hình trong whitelist, mặc định <code>gpt-oss-120b</code> .

Nhà cung cấp	Giao thức truyền	Bộ chọn chat	Vòng lặp tool	Thị giác	Ghi chú
Cerebras	HTTP tương thích OpenAI	Khi có khoá	Có	Chưa kết nối	Dùng chung đường mã với BytePlus.
DeepSeek	HTTP tương thích OpenAI	Không	Có	Không	Chỉ dùng cho tác vụ nền.
Anthropic Claude	Anthropic SDK	Khi có khoá	Không	Không	Chỉ chat văn bản; yêu cầu nó điều khiển tool sẽ báo lỗi rõ ràng.



Hình 4. Định tuyến chat và bộ định tuyến mô hình nội bộ. Bộ định tuyến xác định đúng một mô hình cho mỗi lần gọi; tầng đầu tiên đã được cấu hình sẽ được chọn.

■ Quy tắc định tuyến

- **Mặc định ưu tiên cloud.** Bộ chọn chat chọn sẵn BytePlus nếu đã cấu hình, sau đó đến Cerebras, nếu không thì mô hình cục bộ. Một bản triển khai không có khoá cloud sẽ chạy hoàn toàn trên mô hình cục bộ.
- **Không âm thầm chuyển sang nhà cung cấp khác.** Lỗi từ nhà cung cấp tạo ra một thông báo có mã, bằng ba ngôn ngữ (xác thực, giới hạn tốc độ, quá tải, kết nối, API) và lượt hỏi vẫn được lưu lại. BytePlus và Cerebras thử lại một lần khi gặp 429, 503 hoặc 529 sau một khoảng trễ có thể cấu hình.
- **Ba điểm vào nội bộ.** `callModelText` (không có tool), `callModelChat` (vòng lặp tool) và `callModelGenerate` (JSON có cấu trúc).
- **Kiểm tra năng lực.** Claude có thể tóm tắt nhưng không thể chạy node agent của workflow; lượt chạy thất bại tại node đó với lỗi tương minh thay vì tạo ra một câu trả lời không dùng tool.

• Đầu ra có cấu trúc theo từng nhà cung cấp

Việc soạn workflow cần JSON đáng tin cậy. Mức hỗ trợ khác nhau giữa các nhà cung cấp và mô hình, nên chiến lược được chọn theo từng nhà cung cấp dựa trên số liệu đo: chế độ JSON khi được hỗ trợ; JSON chỉ qua prompt khi mô hình từ chối tham số `response_format`; buộc gọi tool ("structured tool") làm mặc định cho việc suy ra nhánh trên BytePlus `gpt-oss-120b` (10 trên 10 trong thử nghiệm); JSON schema nghiêm ngặt trên Cerebras. Function calling nghiêm ngặt của DeepSeek cho kết quả đo kém hơn và chỉ bật khi chủ động chọn.

• Hạch toán chi phí

Một bảng giá (USD trên một triệu token đầu vào và đầu ra) bao phủ các mô hình Claude, BytePlus, Cerebras và DeepSeek. Mô hình không khớp mục nào được tính theo mức giá ước lượng mặc định ngang Sonnet; các mô hình chưa công bố giá không hiển thị chi phí thay vì một con số phỏng đoán. Token được ghi nhận theo từng tin nhắn chat và từng bước workflow, cùng với mô hình thực sự đã trả lời, và chi phí được tính tại thời điểm ghi. Phần giám sát hiển thị mô hình, tool và chi phí của các lượt chat và lượt chạy workflow bên cạnh các phiên agent được giám sát.

6 Tool và connector

Mọi việc trợ lý có thể làm đều là một tool. Tool đến từ ba nguồn: một tập tool nội bộ khép kín, các connector tích hợp sẵn mà người dùng đã kết nối, và các MCP server do người dùng thêm. Cả ba dùng chung một đường điều phối và một cổng an toàn.

• 6.1 Tool nội bộ (luôn sẵn có)

Nhóm	Tool
Giám sát (9, chỉ đọc)	<code>laam_list_agents</code> , <code>laam_get_agent</code> , <code>laam_query_stats</code> , <code>laam_list_machines</code> , <code>laam_find_stuck</code> , <code>laam_search_sessions</code> , <code>laam_get_timeline</code> , <code>laam_query_audit</code> , <code>laam_metrics_digest</code> . Các tool này đọc kho dữ liệu giám sát của chính LAAM, không phải dữ liệu nghiệp vụ. Chính chín tool này được cung cấp cho agent bên ngoài qua MCP server của LAAM.
Web (2)	<code>web_search</code> qua SearXNG tự host (suy giảm êm nếu dịch vụ ngừng); <code>web_read</code> tải một URL dưới dạng văn bản, đối chiếu host của URL và của mọi bước chuyển hướng với một blacklist tên máy riêng tư và metadata, giới hạn 6,000 ký tự.
Tiện ích (1)	<code>util_calc</code> : tính toán số học tất định bằng bộ phân tích shunting-yard, không dùng <code>eval</code> .

• 6.2 Connector tích hợp sẵn

Mỗi connector tích hợp sẵn triển khai cùng interface `Connector`: một id, một chế độ xác thực, và danh sách tool, mỗi tool khai báo `kind: "read" | "write"` và có an toàn để chạy không giám sát trong workflow hay không. Thông tin xác thực được mã hoá theo từng người dùng (AES-256-GCM với khoá dẫn xuất riêng cho từng người dùng bằng HKDF-SHA256). Việc làm mới token OAuth chạy tại một điểm chặn duy nhất `execute()` dưới advisory lock của Postgres, vì một số nhà cung cấp xoay vòng refresh token chỉ dùng một lần; quyền bị thu hồi sẽ chuyển connector sang trạng thái "cần kết nối lại".

Connector	Tool	Ghi	Xác thực	Ví dụ thao tác ghi
GitHub	8	2	Token	Tạo issue, bình luận

Connector	Tool	Ghi	Xác thực	Ví dụ thao tác ghi
Trello	7	3	Token, tùy chọn uỷ quyền một cú nhấp	Tạo / cập nhật thẻ, bình luận
Jira	6	2	Atlassian OAuth 3LO hoặc API token	Tạo issue, bình luận
Google Drive	5	1	Google OAuth	Tạo thư mục
Google Calendar	4	1	Google OAuth	Tạo sự kiện
Gmail	4	1	Google OAuth	Gửi (xác nhận trong chat; whitelist người nhận trong workflow)
Slack	3	1	Slack OAuth v2	Gửi tin nhắn
WhatsApp	1	1	Cloud API token	Gửi tin nhắn
Zalo	2	1	Zalo OA OAuth v4 (PKCE)	Gửi tin nhắn
Danh mục APILayer	2	0	Một access key	– (9 API trong danh mục)
Danh mục Public-APIs	2	0	Không có	– (8 API không cần khoá, kiểm tra thủ công)
Demo	2	1	Không có	Tạo tác vụ (demo cổng ghi ngoại tuyến)
Tổng	46	14		

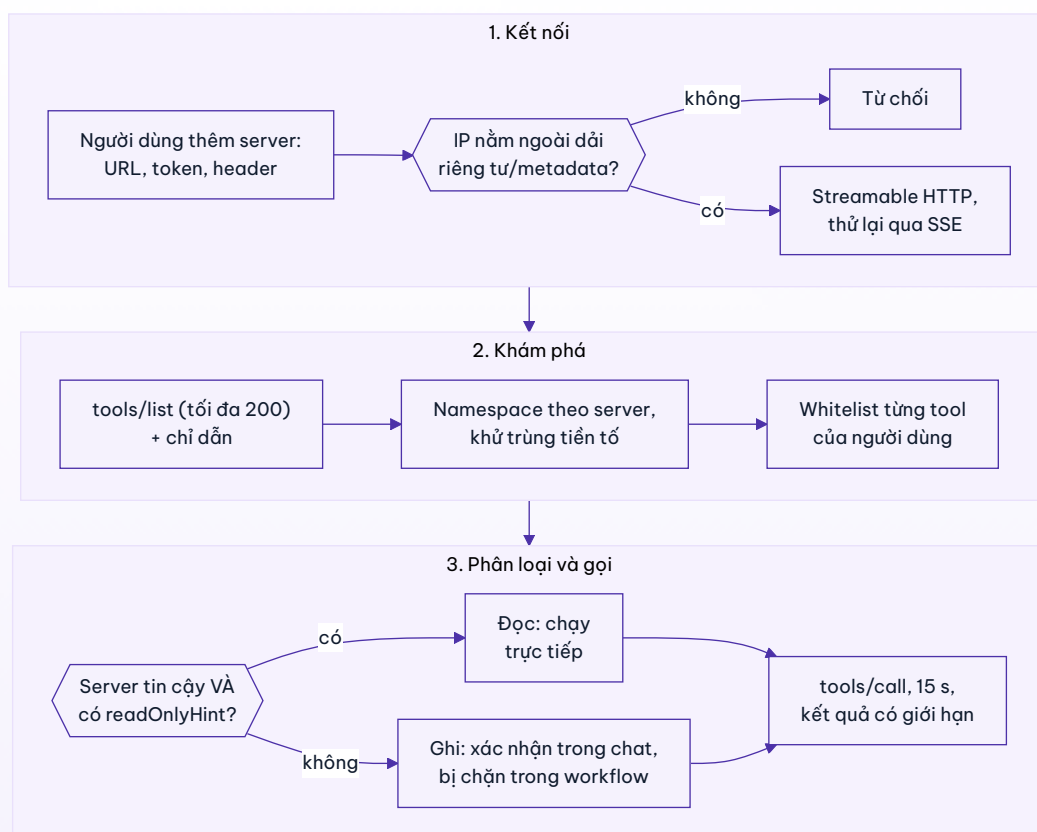
Hai connector danh mục an toàn trước SSRF ngay từ thiết kế: mô hình tìm kiếm trong một danh mục do con người soạn và gọi một mục, và mỗi mục cố định mẫu URL của riêng nó, nên mô hình chỉ có thể yêu cầu những URL do LAAM viết. Các luồng OAuth được ẩn cho đến khi đơn vị vận hành cấu hình client credentials của nhà cung cấp; Jira cũng chấp nhận API token.

6.3 MCP client – đường tích hợp chung

Bất kỳ hệ thống nào hỗ trợ Model Context Protocol đều có thể được người dùng thêm vào lúc runtime: một cổng cơ sở dữ liệu, một server knowledge graph, một endpoint MCP của nhà cung cấp SaaS. Đường MCP là đường chung; một vài chuỗi gửi cho mô hình vẫn nêu tên tool của một knowledge graph server cụ thể (việc dọn dẹp đang được theo dõi). Cấu hình server (URL, bearer token, header) được lưu trong cùng bảng thông tin xác thực mã hoá theo từng người dùng như các connector tích hợp sẵn.

Thuộc tính	Cách triển khai và lý do
Giao thức truyền	Ưu tiên Streamable HTTP, thử lại một lần qua SSE cho các server cũ. Kết nối được mở theo từng thao tác và luôn được đóng.
Guard SSRF	Khi kết nối, tên máy trong URL của server được phân giải DNS và mọi địa chỉ thu được đều được đối chiếu với các dải riêng tư, link-local, metadata, IPv6 unique-local và IPv4-mapped trước khi mở kết nối. Hạng mục gia cố trước production: một guard ghim DNS duy nhất cho mọi yêu cầu ra ngoài, bổ sung dải carrier-grade NAT và kiểm tra lại khi chuyển hướng.
Namespace	Mỗi tool được cung cấp cho mô hình dưới dạng <code>mcp__<server>__<tool></code> , nên các server không thể xung đột với nhau hoặc che khuất tool tích hợp sẵn. Slug của server giữ ổn định khi server được đổi tên hoặc trở sang địa chỉ khác.

Thuộc tính	Cách triển khai và lý do
Khử trùng tiền tố	Khi tên một tool là tiền tố chặt của tên tool khác trên cùng server, tên ngắn hơn được thêm phần đệm. Do trên một nhà cung cấp: tool có tên dài hơn được gọi đúng 0 trên 6 lần trước khi sửa và 6 trên 6 sau khi sửa; trên một server có 49 tool, thay đổi này giúp 11 tool có thể được gọi tới.
Chỉ dẫn của server	Chỉ dẫn trong <code>initialize</code> của từng server được thu lại và đặt vào system prompt, nên thông tin phạm vi mà server công bố không phải suy ra lại bằng các lần gọi liệt kê thêm ở mỗi câu hỏi.
Giới hạn	200 tool mỗi server (ghi log khi cắt bớt), timeout 15 giây mỗi lần gọi, văn bản lỗi gửi cho mô hình giới hạn 300 ký tự, chặn trên 200k ký tự cho dữ liệu thô kèm dấu hiệu cắt bớt hiển thị rõ; định hình mỗi kết quả còn 8k ký tự (cục bộ) hoặc 120k (cloud).
Lỗi trong phản hồi	Cờ <code>isError</code> của MCP được truyền tiếp: runtime của workflow dừng khi gặp cờ này và pipeline soạn bằng AI loại bỏ các quan sát như vậy.



Hình 5. Đường xử lý tool MCP. Việc phân loại mặc định an toàn (fail closed): một tool chỉ được coi là tool đọc khi cả người dùng và server đều xác nhận như vậy.

6.4 Xử lý kết quả: bảng và bản tóm lược do mã dựng

Kết quả MCP đến dưới dạng văn bản chứa JSON, còn tool nội bộ trả về đối tượng. Một hàm bóc tách duy nhất chuẩn hoá cả hai, và mọi bước kiểm tra hình dạng dữ liệu đều dùng hàm này; một guard bỏ qua bước này sẽ âm thầm không bao giờ kích hoạt.

- Bảng dữ liệu.** Khi kết quả có dạng bảng (ít nhất 3 dòng và 2 cột với tập khoá nhất quán, tối đa 500 dòng), mã dựng một bảng và stream nó thành frame `view`; gợi ý biểu đồ được thêm cho tối đa 25 dòng. Sau đó mô hình được báo rằng bảng đã hiển thị trên màn hình. Động lực đến từ số liệu đo: trên 13 câu hỏi thử nghiệm, chỉ 2 kết quả đạt ngưỡng kích thước cũ để tạo bảng; ở 11 câu còn lại, các con

số trên màn hình là do mô hình gõ lại kết quả, và nó đã gây lỗi (tên sản phẩm bị cắt, khoảng trắng thừa trong một định danh, mất một ô và lặp một dòng).

- **Bản tóm lược cho mô hình.** Kết quả lớn đến với mô hình dưới dạng số đếm, các cột, tổng do mã tính và các dòng mẫu. Trong một trường hợp đo được, việc gõ lại một kết quả 62 dòng mất 103 giây và 3,225 token đầu ra, kèm lỗi; bản tóm lược loại bỏ chi phí đó.
- **Quy tắc định dạng.** Bảng Markdown được mã sửa lại; số dùng dấu phân cách hàng nghìn và đơn vị tiền tệ tương minh.

7 Các guard độ tin cậy

Mô hình ngôn ngữ có thể bị đặt, chọn nhầm tool gần giống và bỏ cuộc sớm. LAAM không chỉ dựa vào prompt để ngăn chặn điều này. Mỗi kiểu lỗi quan sát được trong thử nghiệm đều có một guard mang tính cấu trúc trong mã, được kích hoạt bởi các dữ kiện quan sát được (thiếu lần gọi tool, kết quả rỗng, tên sai định dạng) thay vì phân loại ý định của người dùng.

7.1 Vòng xác nhận thao tác ghi

Cổng ghi là guard quan trọng nhất. Mọi lần điều phối đều đi qua `withSafety`, hàm phân loại tool dựa trên chính khai báo của tool. Tool không xác định được coi là tool ghi. Trong chat, một thao tác ghi chưa được xác nhận không bao giờ được thực thi: nó phát ra một tín hiệu mà route chuyển thành lượt hỏi bị tạm dừng.

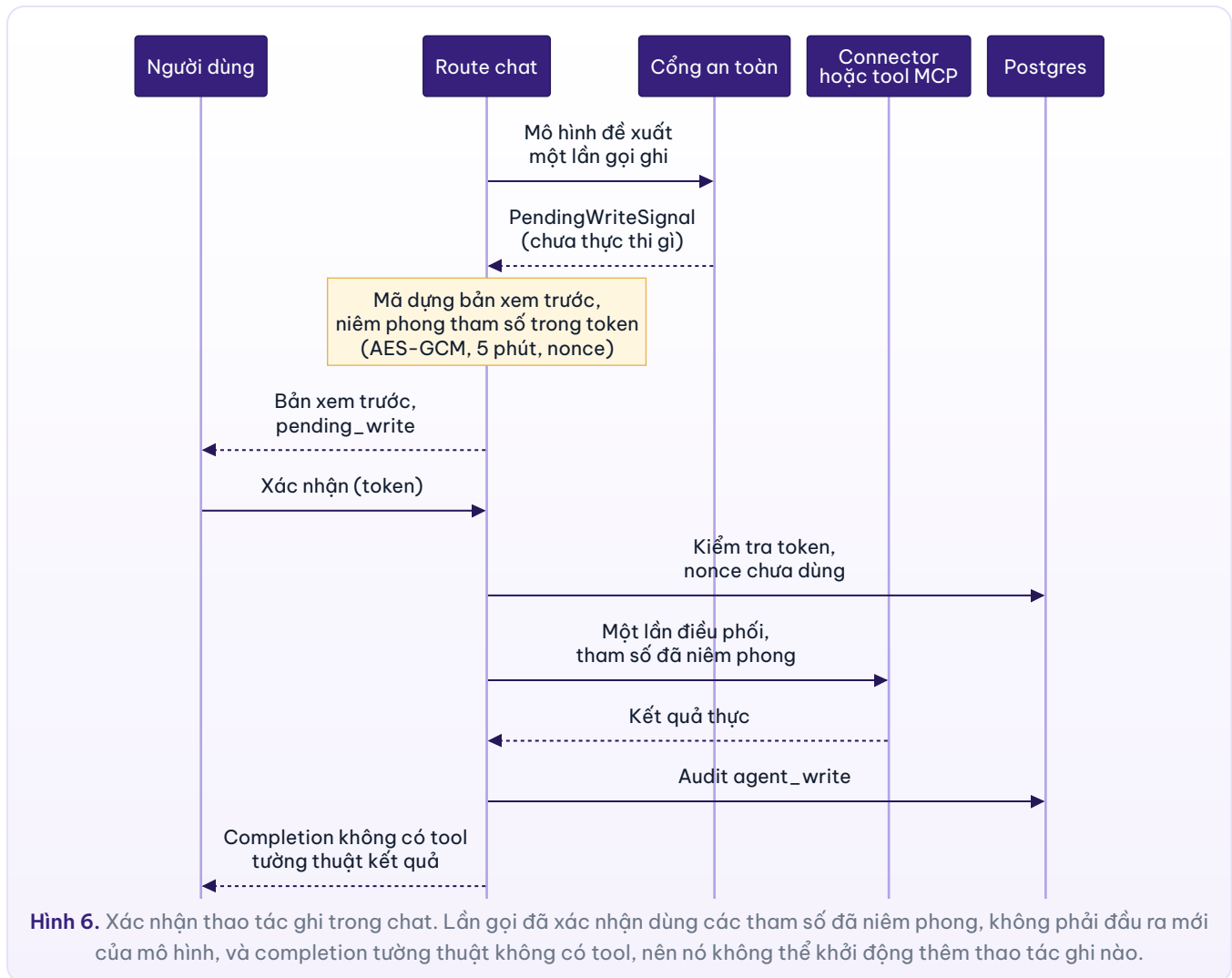
Có ba lựa chọn thiết kế quan trọng ở đây. Bản xem trước hiển thị cho người dùng được mã dựng từ các tham số, không bao giờ từ văn bản của mô hình, nên người dùng phê duyệt đúng nội dung sẽ được gửi. Token không lưu trạng thái (được niêm phong bằng mã hoá có xác thực) và mang theo mô hình ban đầu, nên phần tường thuật sau khi xác nhận tiếp tục với cùng giọng văn. Và nonce của token được đối chiếu với nhật ký audit trước khi điều phối, nên một lần xác nhận đã được ghi nhận sẽ bị từ chối. Hạng mục gia cố trước production: nhận nonce một cách nguyên tử (atomic) và thử lại idempotent cho các thao tác ghi MCP đã xác nhận.

7.2 Guard bám sát dữ liệu

Guard	Điều kiện kích hoạt	Lý do tồn tại
Hỏi lại để bám dữ liệu	Vòng 0, có tool, mô hình trả lời mà không gọi tool. Hỏi lại một lần (có ngoại lệ cho trò chuyện xã giao).	Một mô hình đã bịa ra toàn bộ hồ sơ dự án, kể cả một người phụ trách không tồn tại, thay vì tra cứu.
Rỗng không có nghĩa là không có	Tool trả về 0 dòng hoặc số đếm bằng 0.	Trên một truy vấn tìm bất thường, một cách diễn đạt không trả về gì trong khi ba cách diễn đạt khác trả về 9 đến 18 dòng. Mô hình được báo rằng kết quả rỗng không phải bằng chứng của việc không tồn tại, được cho xem truy vấn đã dùng, và được phép hỏi lại một lần bằng chính câu chữ của người dùng.
Guard chống báo ghi sai	Văn bản người dùng trông như yêu cầu ghi; câu trả lời báo thành công; không có tool ghi nào được thực thi.	Các mô hình cục bộ đã tường thuật những thao tác ghi chưa từng thực hiện. Khi phát hiện ý định ghi, đầu ra được đệm lại và các tuyên bố thành công không có căn cứ bị thay thế.

Guard	Điều kiện kích hoạt	Lý do tồn tại
Định tuyến lại theo ý định tool	Tham số khớp chính xác với đúng một tool khác.	Một mô hình đã gọi một tool có tên là tiền tố của tool dự định, với tham số của tool dự định.
Sửa tên tool	Token định dạng bị lọt vào tên tool.	Định dạng chat của một mô hình open-weight đôi khi làm lọt các ký hiệu kênh và hậu tố vào tên tool.
Ghim câu hỏi	Một lượt có dùng tool trên nhánh cloud (BytePlus hoặc Cerebras).	Một câu trả lời đã trôi sang câu hỏi trước sau một chuỗi tool dài.
Loại bỏ phần thừa	JSON gọi tool ở đầu câu trả lời.	Các câu trả lời đã lưu đôi khi bắt đầu bằng JSON thô.
Nudge chống dùng sai audit	Nhật ký audit của chính LAAM bị truy vấn cho một câu hỏi nghiệp vụ.	Mô hình đọc nhầm nhật ký và kết luận "không có dữ liệu".
Guard buộc lấy dữ liệu PLANNED	Chỉ có các tool cấu trúc hoặc liệt kê được gọi. Tắt mặc định qua cờ cấu hình; cấu hình theo từng bản triển khai.	Một mô hình đã kiểm tra schema nhiều lần, rồi trả lời "không có dữ liệu" mà không lấy dữ liệu nào.
Đi sâu từ danh sách đến chi tiết PLANNED	Sau một tool danh sách đã cấu hình, câu hỏi nêu tên đúng một thực thể từ kết quả của nó. Tắt mặc định qua cờ cấu hình.	Mã gọi tool chi tiết đi kèm. Các cặp tool là cấu hình, nên không có tên connector nào bị hard-code. Trong thử nghiệm, sau đó 12 trên 12 câu hỏi chi tiết bằng giọng nói đã đến được tool chi tiết.

Các guard chỉ kích hoạt một lần mỗi lượt và suy giảm êm: không guard nào có thể chặn vĩnh viễn một câu trả lời, và tín hiệu ghi luôn được truyền qua chúng.



8 Engine workflow

Một workflow là một graph JSON gồm các node có kiểu, lưu theo từng người dùng. Workflow có thể chạy theo yêu cầu, theo lịch cron, hoặc chạy thử (dry-run). Các lượt chạy được ghi nhật ký từng bước nên vẫn tồn tại qua một lần khởi động lại tiến trình, và các thao tác ghi được lưu trong một sổ cái idempotency nên một lượt chạy được tiếp tục không bao giờ gửi hai lần.

8.1 Mô hình graph

Loại node	Hành vi
agent	Chạy vòng lặp tool chỉ với các tool đọc nội bộ của LAAM, sau đó một lần gọi cuối qua bộ định tuyến nội bộ. Tùy chọn preset agent tùy chỉnh (preset bị thiếu sẽ báo lỗi rõ ràng) và tùy chọn schema đầu ra. Chế độ <code>extract</code> lấy một trường từ bước trước bằng mã, không gọi mô hình; chế độ này được thêm vì việc trích xuất bằng mô hình chiếm toàn bộ chi phí của một số lượt chạy (khoảng 3 s mỗi lần gọi so với 24–55 ms cho mỗi lần gọi MCP).
connector	Gọi một hành động của connector tích hợp sẵn với tham số dạng template (<code>{{steps.x.output}}</code> , <code>{{trigger}}</code>).

Loại node	Hành vi
mcp	Gọi một tool trên MCP server của người dùng. Tùy chọn <code>pollUntil</code> gọi lại một lệnh kiểm tra trạng thái cho đến khi một trường đạt giá trị mong muốn; tùy chọn này do mã suy ra từ các lần gọi đã quan sát, không bao giờ do mô hình soạn.
condition	Đánh giá một vị từ (10 toán tử, có thể kết hợp bằng <code>all / any</code>) và đi theo cạnh <code>true</code> hoặc <code>false</code> .
foreach	Chạy một graph con cho từng phần tử với <code>{{item}}</code> và <code>{{index}}</code> , lần lượt từng phần tử.

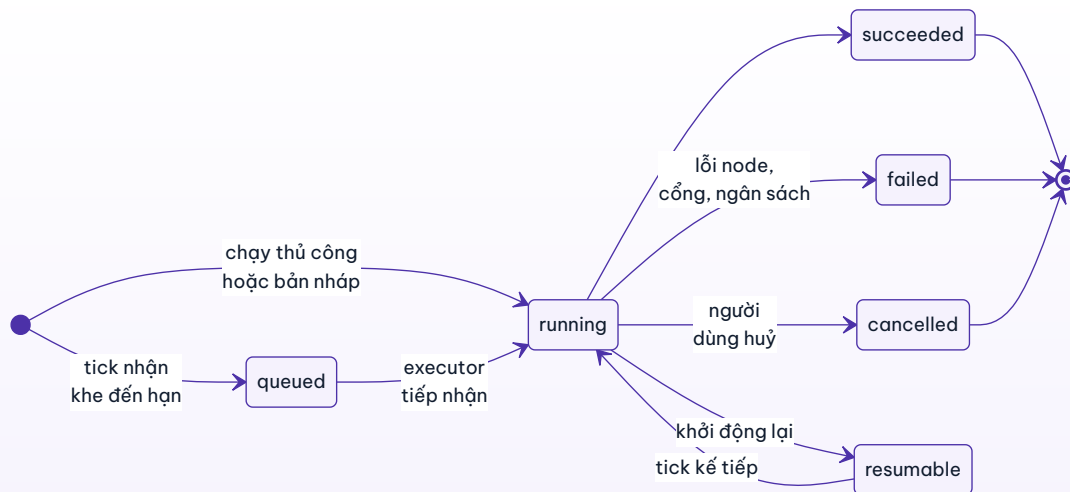
Dữ liệu bên ngoài chỉ đi vào workflow qua các node connector và MCP. Các loại node được cố định trong mã. Ngân sách: 200 bước và 100 phần tử for-each mỗi lượt chạy; ở chế độ song song, mỗi node rẽ ra tối đa 12 cạnh.

8.2 Chế độ thực thi

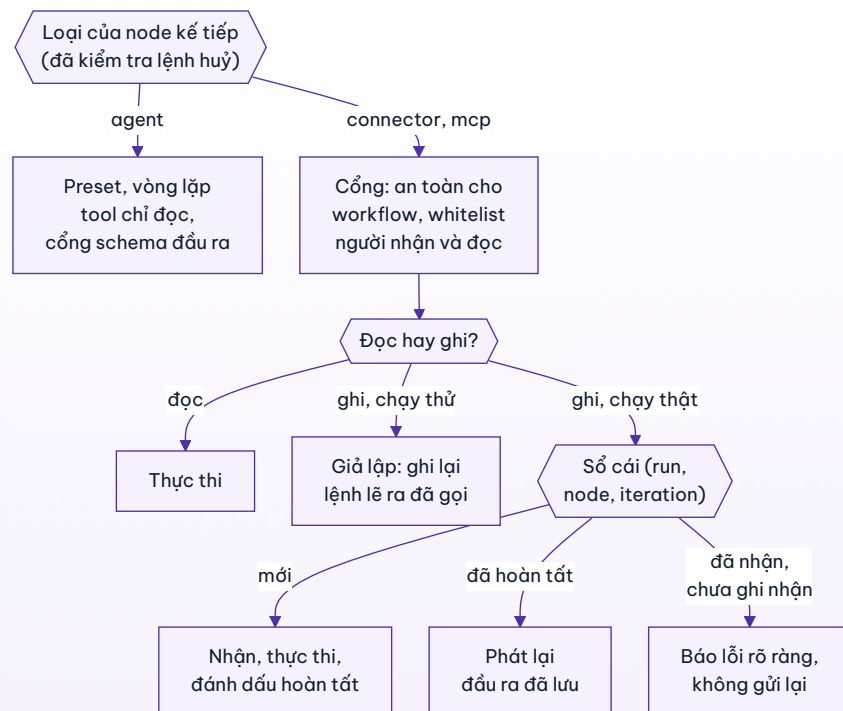
- **Tuần tự** (mặc định): một bộ duyệt cạnh đệ quy đánh giá điều kiện, chạy thân for-each, dùng chung một bộ đếm ngân sách và dừng ở lỗi đầu tiên.
- **Song song** (cờ theo từng graph): một bộ lập lịch DAG theo thứ tự tô-pô với rẽ nhánh (fan-out) và hội tụ kiểu OR (fan-in), nhiều node bắt đầu, và cắt tia ba trạng thái cho các nhánh không được chọn. Hai semaphore tách công việc phụ thuộc mô hình (2 đồng thời) khỏi I/O (6 đồng thời). Dừng khi lỗi, hủy và ngân sách đều mang tính hợp tác: một cờ hủy, sau đó chờ các node đang chạy hoàn tất trước khi kết thúc. Một bộ phát hiện deadlock bảo vệ bộ lập lịch, và một golden test chứng minh rằng graph tuần tự ở mức đồng thời 1 tạo ra lượt chạy giống hệt từng byte so với bộ duyệt tuần tự.
- **Kiểm tra hợp lệ** dùng thuật toán Kahn để phát hiện chu trình (một bản kiểm tra theo chiều sâu trước đây đã báo nhầm các hình thoi), kiểm tra khả năng tiếp cận từ các node bắt đầu, và một nhóm quy tắc lint từ chối các tham chiếu mà một node thực tế không thể tạo ra. Cùng bước kiểm tra này chặn thao tác Lưu.

8.3 Vòng đời lượt chạy, độ bền vững và ghi idempotent

- **Snapshot.** Graph được sao chép vào dòng dữ liệu của lượt chạy khi bắt đầu, nên việc chỉnh sửa workflow không bao giờ làm thay đổi một lượt chạy đang diễn ra.
- **Nhật ký.** Mỗi node ghi một dòng `workflow_run_step` (đầu vào, đầu ra có giới hạn, token, chi phí và mô hình thực sự được dùng) và phát một sự kiện bước kèm bản xem trước đầu ra 280 ký tự cho canvas trực tiếp.
- **Ghi idempotent.** Trước một thao tác ghi, executor nhận một dòng có khoá (run, node, iteration) với ràng buộc unique. Sau khi thành công, dòng được đánh dấu hoàn tất kèm đầu ra. Khi tiếp tục, thao tác ghi đã hoàn tất sẽ phát lại đầu ra đã lưu; thao tác ghi đã nhận nhưng chưa ghi nhận kết quả sẽ báo lỗi rõ ràng thay vì mạo hiểm gửi trùng email hoặc tin nhắn.
- **Tiếp tục sau sự cố.** Khi khởi động, các lượt chạy vẫn được đánh dấu running chuyển thành resumable. Tick kế tiếp của bộ lập lịch nhận tối đa 25 lượt với `FOR UPDATE SKIP LOCKED`, dựng lại ngữ cảnh từ nhật ký và bỏ qua các node đã hoàn tất.
- **Cổng kiểm tra đầu ra.** Node agent có schema đầu ra được kiểm tra các khoá bắt buộc bị thiếu (null được tính là thiếu) và sai kiểu; node có một lần tự sửa với tên trường chính xác, sau đó thất bại tại node đó. Việc kiểm tra kiểu tồn tại vì từng có lần một bản kiểm tra chỉ xét sự hiện diện đã để lọt một danh sách kết quả rỗng và một lượt chạy kết thúc toàn màu xanh mà không có dữ liệu.
- **Cổng người nhận.** Các thao tác ghi gửi ra ngoài tổ chức (email, Slack, WhatsApp, Zalo) yêu cầu mọi người nhận đã phân giải phải nằm trong whitelist của kênh đó; whitelist rỗng sẽ chặn. Bộ phân tích người nhận từ chối header injection.



Hình 7. Các trạng thái lượt chạy workflow. Lượt chạy bị gián đoạn do tiến trình khởi động lại sẽ chuyển sang resumable và chạy tiếp từ nhật ký ở tick kế tiếp; lượt chạy đã huỷ không bao giờ được tiếp tục.



Hình 8. Thực thi theo từng node. Mọi bước đều được ghi nhật ký; mọi thao tác ghi đều đi qua sổ cái idempotency.

8.4 Trigger và bộ lập lịch

Trigger	Đường xử lý
manual	POST /api/workflows/{id}/run , tùy chọn {dryRun:true} : thao tác đọc chạy thật, thao tác ghi được giả lập.
draft	Cùng endpoint với một graph chưa lưu từ khung soạn bằng AI. Chỉ chấp nhận với dryRun:true , sau đó được chuẩn hoá và kiểm tra hợp lệ như một graph đã lưu.
schedule	Cron năm trường cho mỗi lịch. POST /api/workflows/tick nhận các lịch đến hạn một cách nguyên tử (chèn lượt chạy và đẩy thời điểm chạy kế tiếp trong cùng một transaction, unique theo từng khe lịch), sau đó thực thi các lượt chạy đang chờ và có thể tiếp tục. Các khe bị lỡ chỉ chạy một lần và được ghi nhận vào bộ đếm (missedCount) thay vì chạy bù.

Endpoint tick được một bộ lập lịch bên ngoài trên host gọi mỗi phút một lần và xác thực bằng secret dùng chung (so sánh thời gian hằng); không có bộ hẹn giờ trong tiến trình. Lướt chạy có thể bị huỷ bất cứ lúc nào; engine nhận biết lệnh huỷ ở lần kiểm tra trước node kế tiếp. Các trạng thái kết thúc tạo ra một thông báo trong ứng dụng kèm liên kết trực tiếp. Có sẵn sáu template đã bản địa hoá (bản tổng hợp qua đêm, cảnh báo agent bị kẹt, phân loại theo lịch, bản tổng hợp được kiểm chứng bởi mô hình chấm, báo cáo đa nguồn gửi qua email, và một bản demo) cùng thao tác lưu thành template.

9 Soạn workflow bằng AI

BUILT Người dùng có thể mô tả một quy trình tự động hoá bằng ngôn ngữ thông thường và để LAAM dựng nó. Pipeline soạn thảo **bám theo vết thực thi**: mô hình trước hết phải quan sát các lần gọi tool thực trên chính connector của người dùng, và chỉ được dựng graph từ những lần gọi mà nó đã thực sự thấy thành công. Mã, không phải mô hình, biên dịch graph cuối cùng.

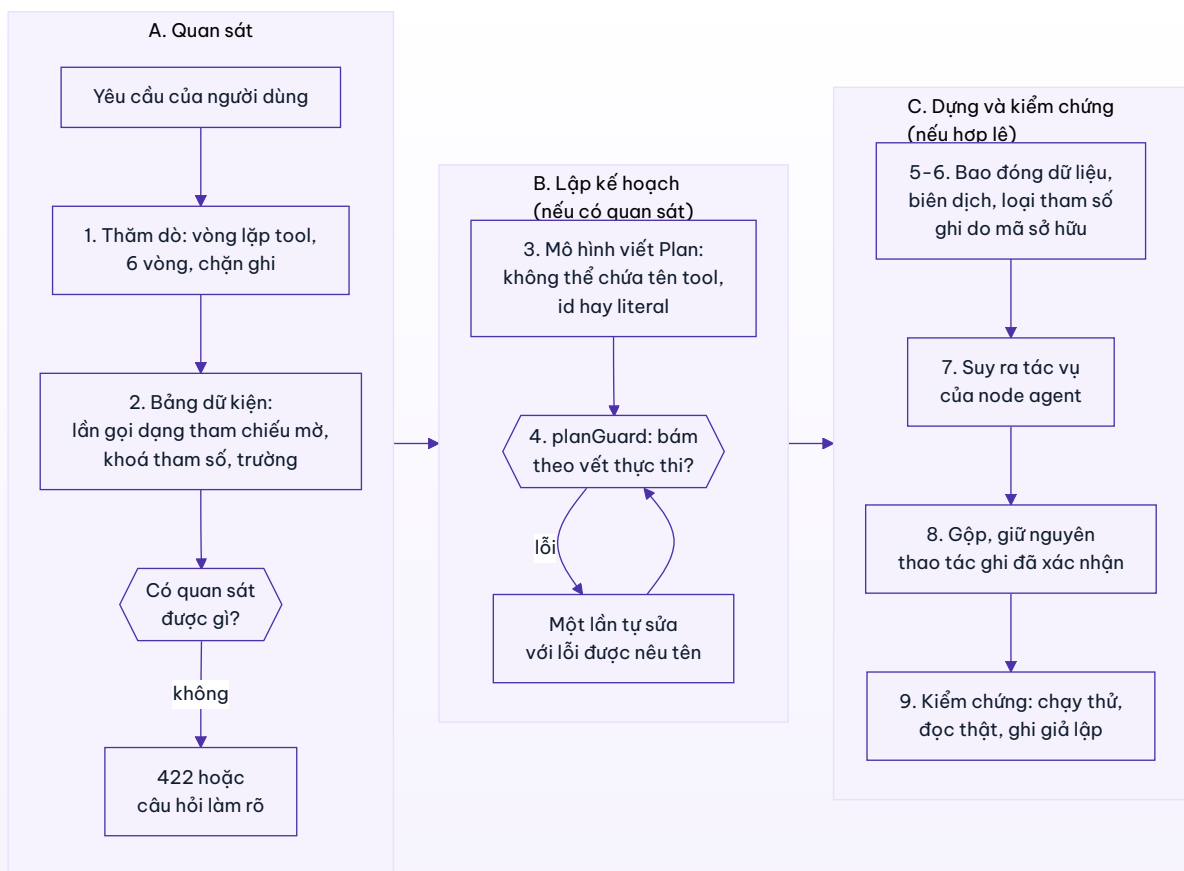
Giai đoạn	Điều được bảo đảm
Thăm dò	Mô hình được thấy tên tham số nhưng không bao giờ thấy giá trị thực, nên không thể sao chép một định danh đang dùng vào graph đã lưu (một lỗi trước đây đã làm đúng như vậy). Thao tác ghi bị chặn và được ghi lại, không bao giờ được thực thi.
Plan DSL	Schema của kế hoạch không thể biểu diễn tên tool, id connector hay giá trị tham số dạng literal. Node của kế hoạch tham chiếu các lần gọi đã quan sát bằng tham chiếu mờ và nối tham số tới các bước trước.
planGuard	Từ chối ngay ở lỗi đầu tiên, nêu tên các lựa chọn thực để thông báo lỗi đồng thời là prompt sửa: tham chiếu phải tồn tại; khoá tham số phải có thật; giá trị phải dạng template, không phải literal trần; nguồn phải bám theo dữ liệu đã quan sát; trường được trích xuất phải tồn tại trong payload đã quan sát; rẽ nhánh ≤ 12 ; không có thao tác đọc thừa.
Bao đóng dữ liệu	Kéo vào các lần gọi tạo dữ liệu mà kế hoạch phụ thuộc, kể cả các lần gọi từ những lướt trước của cuộc hội thoại soạn thảo. Nếu không có bước này, "phần tử đầu tiên" từng bị cố định vĩnh viễn vào một phần tử cụ thể.
Biên dịch	Literal có entropy cao trở thành tham chiếu bước; tham số người nhận và nội dung của thao tác ghi bị loại bỏ và được khẳng định lại do mã sở hữu; tính song song được suy ra.
Gộp	Các node ghi đã được con người xác nhận (id, tham số, người nhận) giữ nguyên qua các lần dựng lại; prompt đã chỉnh tay được giữ lại khi danh sách bước chỉ được nối dài thêm.
Kiểm chứng	Chạy thật graph với thao tác ghi được giả lập (40 bước, 2 phần tử for-each, hạn chót 60 giây) và gắn nhãn cho các node lỗi trên canvas.

• Hai bề mặt soạn thảo

- **Sinh một lần** (`POST /api/workflows/generate`): thăm dò, lập kế hoạch, kiểm chứng; trình soạn áp dụng kết quả như một đề xuất có thể hoàn tác sau thẻ xác nhận, kể cả khi một số node báo đỏ, để người dùng thấy chính xác phần nào thất bại.
- **Khung soạn thảo nhiều lướt** (`POST /api/workflows/generate/chat`): mỗi lướt chạy một vòng lặp tool bị chặn ghi, sau đó chạy song song câu trả lời dạng văn bản với việc dựng kế hoạch (tiết kiệm trung vị 2.6 s mỗi lướt). Một cổng không-có-dữ-liệu từ chối vẽ graph nếu không có thao tác đọc nào thực sự trả về câu trả lời, và thay vào đó đặt câu hỏi làm rõ. Chỉ graph đã được kiểm chứng toàn xanh mới được đề xuất, kèm tối đa ba dòng mẫu đã quan sát. Thao tác ghi được nối thêm trong một bước riêng do con người xác nhận. **Chạy thử** thực thi graph đề xuất như một lướt chạy `draft` và tóm lược kết quả

trở lại cuộc hội thoại với các giá trị có dạng định danh bị che, để mô hình có thể phản ứng với kết quả thực (ví dụ thêm nhánh cho trạng thái "cần làm rõ") mà không hard-code giá trị.

- **Rà soát** (`POST /api/workflows/review`): rà soát bằng ngôn ngữ thông thường đối với graph đang có trong trình soạn.



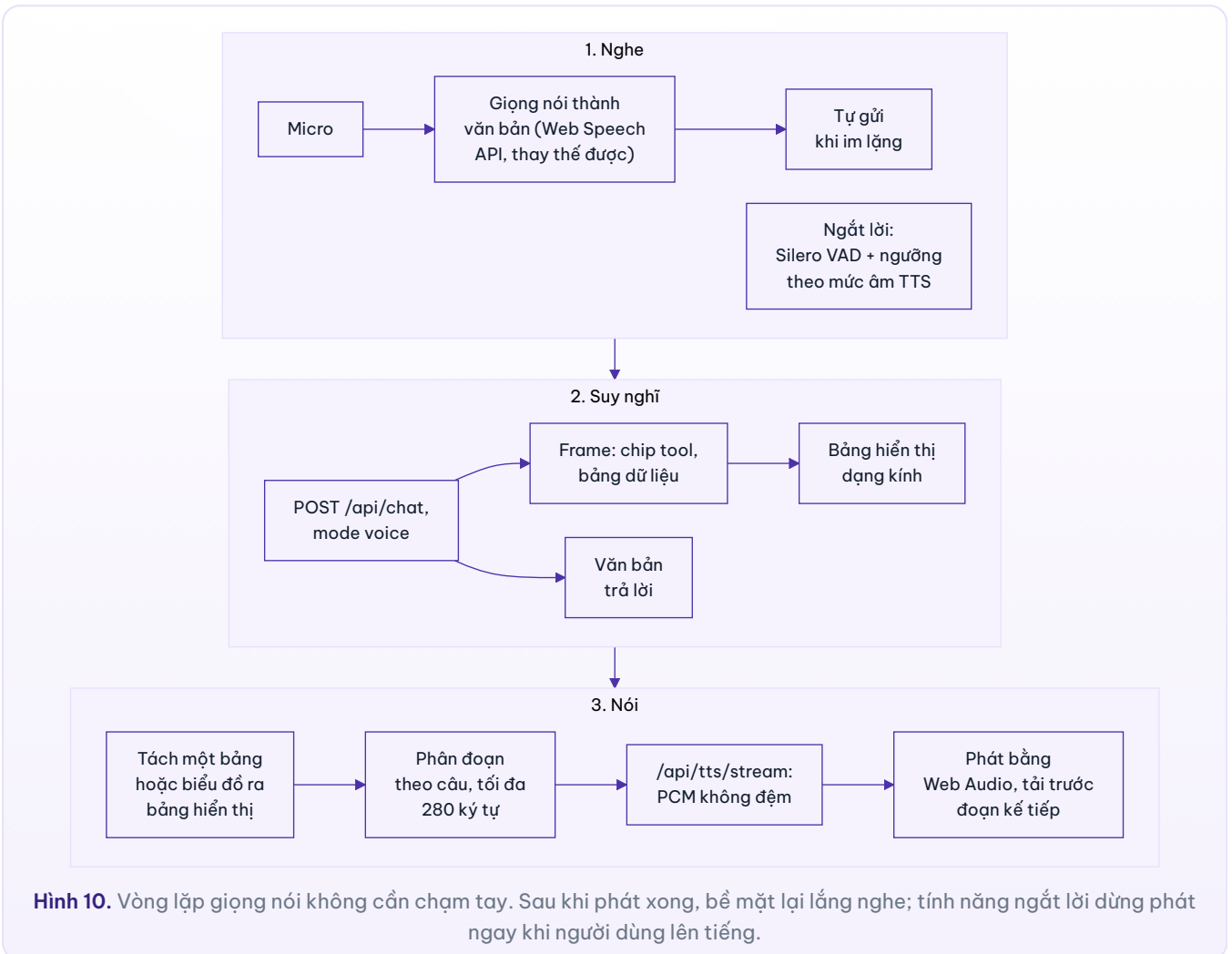
Hình 9. Soạn thảo bám theo vết thực thi. Nếu kế hoạch đã sửa vẫn thất bại, pipeline chuyển sang bộ biên dịch vết cũ hơn và thông báo rõ điều đó.

10 Pipeline giọng nói

Bề mặt giọng nói là một trung tâm điều khiển toàn màn hình chạy cùng engine chat ở chế độ không cần chạm tay. Các agent tùy chỉnh nằm trên vòng trong và các nhóm tool nằm trên vòng ngoài; connector chưa kết nối được làm mờ. Giọng nói vào và ra đều được stream để trợ lý bắt đầu nói trước khi toàn bộ câu trả lời được tổng hợp xong.

- **Cùng engine, khác hướng dẫn.** Lướt giọng nói dùng route chat với `mode: "voice"`. Hướng dẫn giọng nói yêu cầu văn phong nói không có định danh, và nêu rõ sự ngắn gọn không được làm giảm độ sâu tra cứu. Hướng dẫn này nhắm vào một vấn đề đo được: 3 trên 17 lượt giọng nói trả lời hỏi hột hoặc bịa đặt, so với 0 trên 6 ở chế độ văn bản. Trong lần kiểm tra lại với 12 lượt, không lượt giọng nói nào bỏ qua lần gọi tool; 2 lượt vẫn hỏi hột do mô hình chọn sai tool, vấn đề mà cơ chế đi sâu từ danh sách đến chi tiết (đang tắt qua cờ cấu hình) giải quyết.
- **Chuyển giọng nói thành văn bản.** Một interface nhà cung cấp có thể thay thế; bản triển khai đầu tiên là Web Speech API của trình duyệt. Nhà cung cấp Whisper tự host đang ở trạng thái **PLANNED**.
- **Ngắt lời.** Hai cổng: một mô hình phát hiện giọng nói được phục vụ từ chính LAAM, và một ngưỡng động gắn với mức âm lượng phát hiện tại để giọng nói của chính trợ lý không kích hoạt việc ngắt lời.

- **Chuyển văn bản thành giọng nói.** Dịch vụ VieNeu TTS tự host (Python, CPU) với một giọng tiếng Việt có sẵn và một giọng tiếng Anh nhân bản. Endpoint streaming truyền PCM 48 kHz không đệm; việc phân đoạn theo câu giữ mọi yêu cầu tổng hợp nằm trong timeout của route bất kể câu trả lời dài bao nhiêu, và đoạn kế tiếp được tải trong khi đoạn hiện tại đang phát. Giới hạn CPU của container được đặt kèm giới hạn số luồng tương ứng; chỉ giới hạn CPU đã làm thông lượng sụt giảm mạnh. Tính năng tổng hợp giọng nói có sẵn của trình duyệt là phương án dự phòng.
- **Bảng hiển thị.** Mô hình có thể đưa tối đa một bảng hoặc biểu đồ vào mỗi câu trả lời giọng nói; nó được tách khỏi văn bản nói và hiển thị trên một bảng không chặn thao tác. Bảng kết quả được thu gọn mặc định trên cả bề mặt chat và giọng nói.

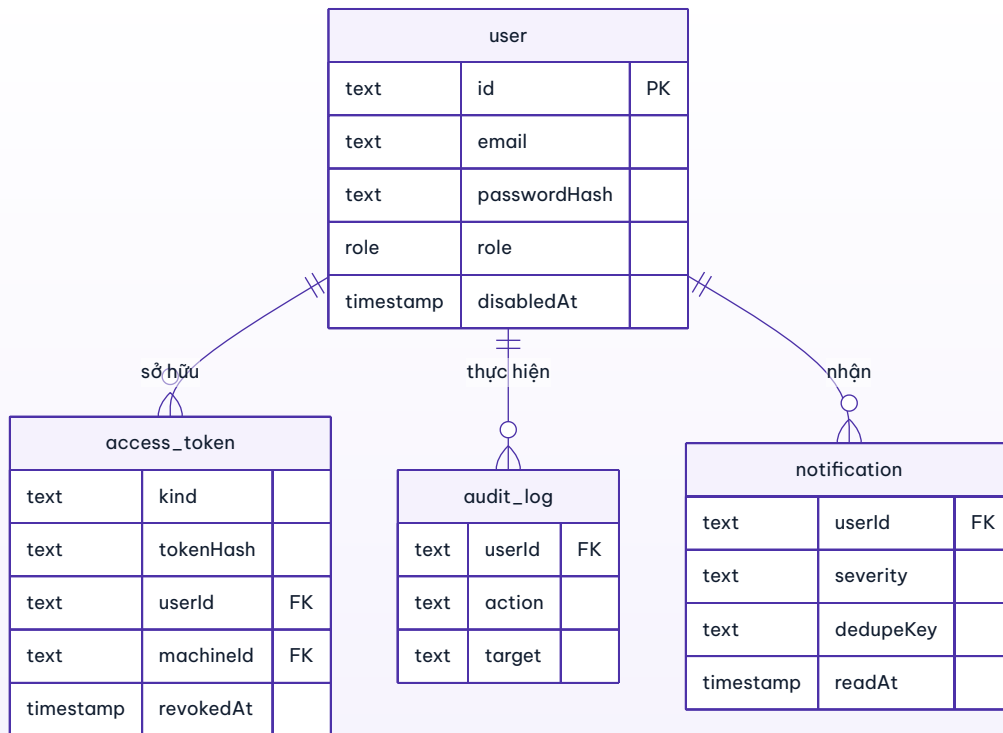


11 Mô hình dữ liệu

LAAM lưu mọi thứ trong một cơ sở dữ liệu PostgreSQL 16: 22 bảng và một enum, được quản lý bởi 19 migration Drizzle có phiên bản (chỉ dùng migration; không bao giờ dùng schema push trên dữ liệu thực). Mô hình là một tổ chức duy nhất có phân vai trò, không phải multi-tenant; dữ liệu theo từng người dùng được cô lập bằng khoá ngoại tới `user` và kiểm tra quyền sở hữu trong mọi truy vấn.

11.1 Định danh và quản trị

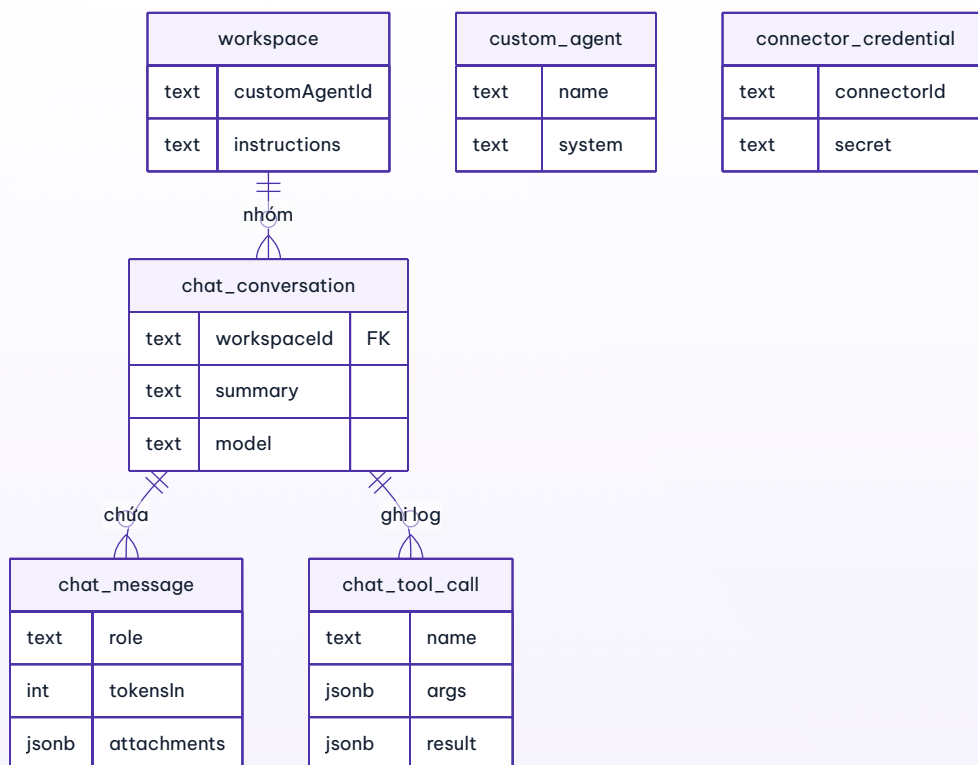
Token truy cập là chuỗi ngẫu nhiên entropy cao, lưu dưới dạng băm SHA-256 và chỉ hiển thị một lần. Một bảng phục vụ ba loại: token của collector (gắn với một máy), token API cá nhân và token MCP. Vô hiệu hoá một người dùng là một transaction: đặt `disabledAt`, thu hồi mọi token, ghi một mục audit. Thông báo được khử trùng lặp bằng một partial unique index trên (user, dedupe key).



Hình 11. Định danh và quản trị. Đăng nhập dùng phiên JWT; các bảng adapter của Auth.js (`account` , `session` , `verificationToken`) tồn tại để tương thích và không được thể hiện trên hình.

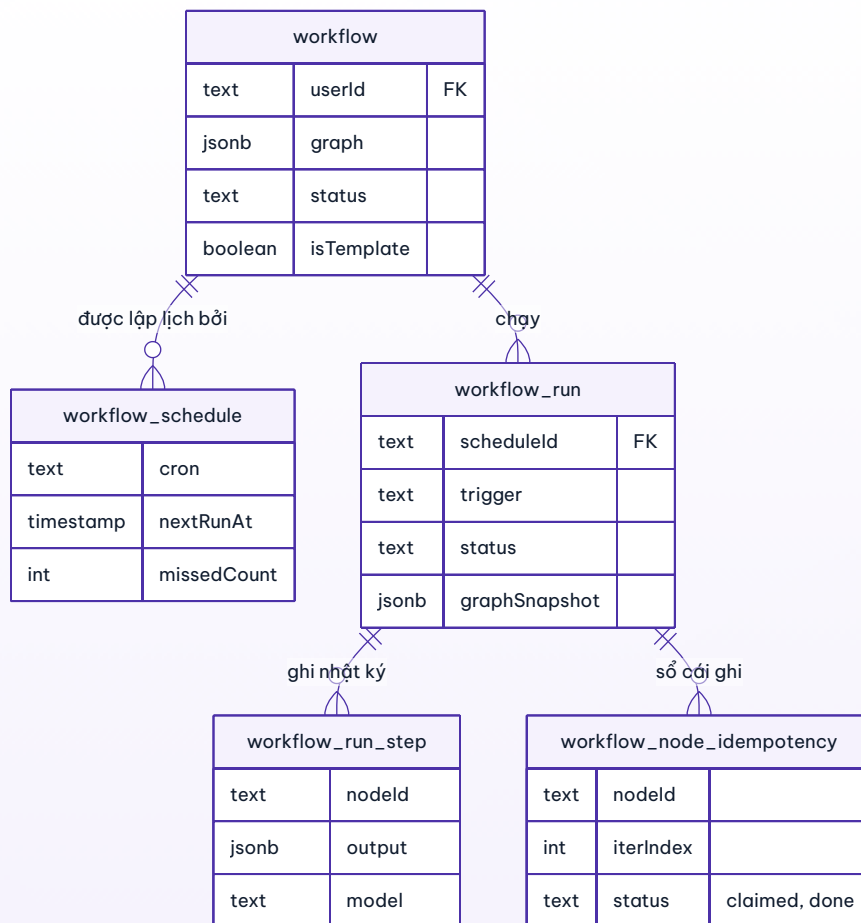
11.2 Chat, agent và connector

`connector_credential` lưu một blob mã hoá cho mỗi cặp (user, connector) và cũng lưu cấu hình MCP server của từng người dùng dưới khoá `mcp:<slug>`. Các blob được mã hoá bằng khoá riêng của từng người dùng, nên một blob bị sao chép sang dòng dữ liệu của người dùng khác sẽ không giải mã được. Tham chiếu tới agent tùy chỉnh từ workspace và node workflow được cố ý để ở dạng mềm (không có khoá ngoại) để preset bị xoá được phát hiện và báo cáo lúc chạy thay vì xoá lan truyền.



Hình 12. Chat, agent và connector. Workspace, hội thoại, agent tùy chỉnh và thông tin xác thực connector đều thuộc về một **user** (hình 11). Kết quả thô của tool được giữ trong **chat_tool_call** để các bảng có thể được dựng lại khi nạp lại.

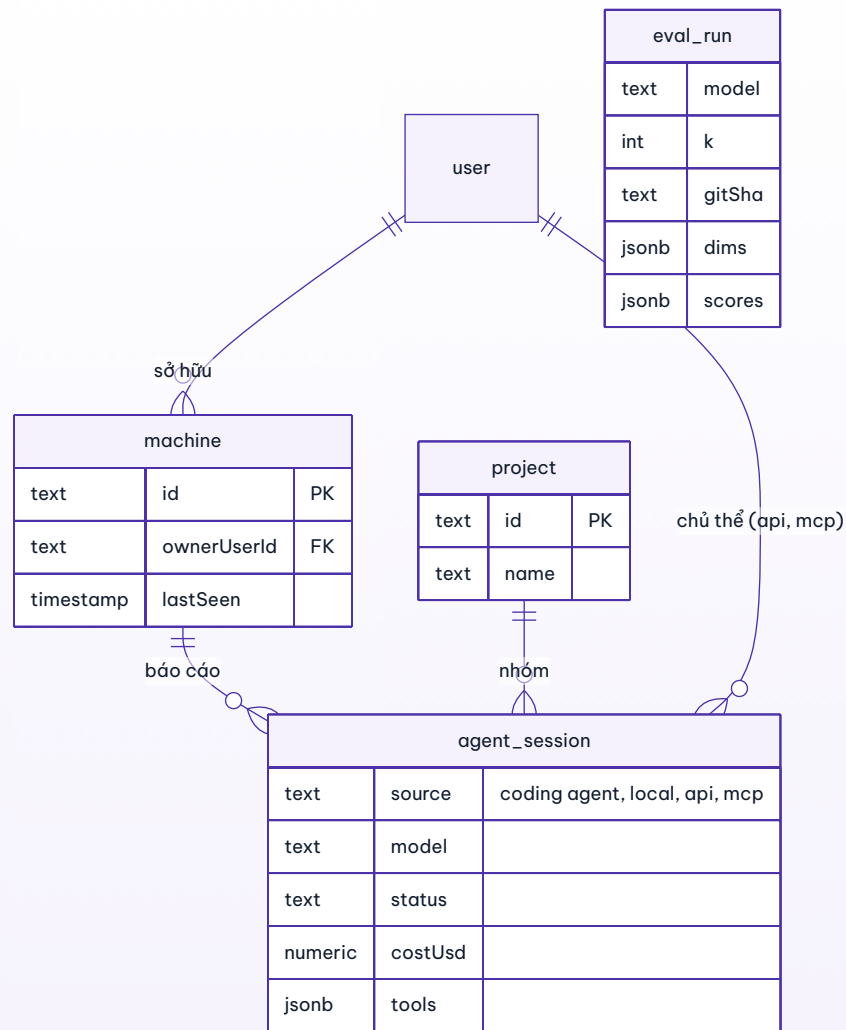
11.3 Workflow



Hình 13. Workflow. Một lượt chạy theo lịch tham chiếu tới lịch của nó qua `scheduleId`. Các ràng buộc unique trên (schedule, scheduled-for) và trên (run, node, iteration) làm cho việc nhận lịch và thao tác ghi có tính idempotent.

11.4 Giám sát và đánh giá

Phiên agent được dẫn xuất từ transcript của coding agent được chia sẻ trong toàn tổ chức theo thiết kế; phiên được tạo qua API hoặc MCP server của LAAM được giới hạn trong phạm vi chủ thể của nó. Mỗi máy xác thực collector của mình bằng một `access_token` loại `collector` (hình 11). Sáu index trigram (GIN) phục vụ tìm kiếm toàn cục; so khớp trigram được chọn thay cho tìm kiếm toàn văn vì PostgreSQL không có sẵn cấu hình tìm kiếm văn bản cho tiếng Việt hoặc tiếng Trung. Danh sách thuộc tính trong cả bốn sơ đồ đã được rút gọn, chỉ giữ các trường có ý nghĩa về mặt kiến trúc.



Hình 14. Giám sát và đánh giá. `eval_run` đứng độc lập: mỗi dòng là một lượt đánh giá ngoại tuyến, được vẽ biểu đồ trên trang Reliability.

12 Bề mặt API

HTTP API gồm 64 tệp route và 88 handler phương thức dưới `/api`. API phục vụ giao diện LAAM; ba endpoint được thiết kế cho bên gọi là máy (nạp dữ liệu từ collector, MCP server và tick của workflow). Các route nghiệp vụ làm thay đổi trạng thái yêu cầu vai trò được phép thay đổi dữ liệu (member trở lên); một số route cá nhân và route trợ giúp (soạn bằng AI, phân tích tài liệu, thông báo, hồ sơ của chính mình) chỉ cần phiên đăng nhập. Các route quản trị yêu cầu owner hoặc admin.

Nhóm	Route	Phạm vi	Xác thực
Xác thực và người dùng	7	Handler của Auth.js, đăng ký (theo chế độ, có giới hạn tốc độ), quản trị người dùng, tự quản lý hồ sơ, token truy cập.	Công khai / phiên / owner-admin
Chat và trợ lý	21	Lượt chat và xác nhận, danh mục mô hình và tool, hội thoại, workspace, agent tùy chỉnh, phân tích tài liệu (OCR, PDF, DOCX), tải URL có guard, proxy TTS, proxy địa lý và thời tiết.	Phiên; thay đổi cần member trở lên

Nhóm	Route	Phạm vi	Xác thực
Connector và MCP	5	Danh sách connector, kết nối / ngắt kết nối / kiểm tra / OAuth, MCP server theo từng người dùng và thăm dò, MCP server của chính LAAM.	Phiên; MCP server bằng token
Giám sát	11	Nạp dữ liệu từ collector, đồng bộ cục bộ, luồng sự kiện SSE, thống kê dashboard, giám sát thống nhất, dòng thời gian phiên, máy, số liệu host, tìm kiếm, cấu hình runtime.	Phiên; nạp dữ liệu bằng token
Workflow	17	CRUD, chạy, nhân bản, template, lượt chạy và hủy, lịch, tick, AI sinh / chat / rà soát / nối thêm thao tác ghi.	Phiên; tick bằng secret
Thông báo	3	Liệt kê, đánh dấu đã đọc, đánh dấu tất cả đã đọc.	Phiên (chỉ dòng của chính mình)

■ Quy ước

- **Quyền sở hữu.** Tài nguyên theo từng người dùng được lọc theo id của người gọi ngay trong truy vấn, nên bản ghi của người dùng khác được xử lý như không tồn tại thay vì bị cấm truy cập.
- **Kiểm tra đầu vào.** Body của request được kiểm tra bằng Zod hoặc bộ phân tích tường minh trước mọi thao tác I/O; id mô hình Claude được đối chiếu với whitelist.
- **Không bao giờ trả lại secret.** Phản hồi về thông tin xác thực và MCP server che secret, chỉ để lộ bốn ký tự cuối; token chỉ hiển thị một lần khi tạo.
- **Streaming.** Lượt chat stream văn bản có frame (chương 4); cập nhật trực tiếp cho phiên, bước workflow và thông báo dùng SSE trên `/api/events`, lọc theo từng người dùng.
- **MCP server.** `POST /api/mcp` triển khai `initialize`, `tools/list`, `tools/call` và `ping` dưới dạng JSON-RPC viết tay qua Streamable HTTP, chỉ cung cấp các tool chỉ đọc. Không thao tác ghi nào của connector có thể được gọi qua đây.

Danh sách endpoint đầy đủ nằm trong phụ lục.

13 Bộ công cụ đánh giá

Ngoài unit test, LAAM có một bộ công cụ đánh giá ngoại tuyến chạy system prompt thật và vòng lặp tool thật với các mô hình thật, chấm điểm hành vi bằng các bộ chấm tự động, và ghi mỗi lượt chạy vào cơ sở dữ liệu để theo dõi xu hướng.

■ Phương pháp

- **Phần chạy thật.** `buildSystemPrompt` và `runToolRounds` của production, cùng một mô hình thật từ nhà cung cấp đã cấu hình (BytePlus, Cerebras hoặc Ollama; thiếu khoá sẽ báo lỗi thay vì chuyển sang phương án khác).
- **Phần giả lập.** Việc điều phối tool. Tool trả về phản hồi giả lập tất định, nên không có connector hay MCP server nào được gọi và các lượt chạy có thể tái lập. Các guard đang tắt qua cờ cấu hình (đi sâu đến chi tiết) không được bật.
- **Bộ chính.** 17 kịch bản bao gồm chọn tool, tham số, kiểm chế (không gọi tool khi trò chuyện xã giao), hiển thị phong phú, ý định ghi, kết thúc, nghiên cứu web và tiện ích. Mỗi kịch bản chạy k lần; 8 bộ chấm cho điểm 7 khía cạnh. Mỗi lượt chạy được lưu trong `eval_run` kèm git SHA và hiển thị trên trang Reliability.

- **Bộ đo quy mô.** Đo tỷ lệ chọn đúng tool khi số tool được cung cấp tăng lên (4, 16, 32, 60), bổ sung bằng một nhóm tool gây nhiễu lấy từ tập hợp tool thật, bao gồm snapshot của một MCP server có 48 tool, với khoảng tin cậy Wilson 95% và tách riêng trường hợp không gọi với gọi sai.

• **Kết quả đo gần nhất**

Bộ chính, ngày 2026-09-03: `gpt-oss-120b` trên BytePlus, k = 5, 17 kịch bản, 85 lượt chạy, điều phối giả lập.

Chọn tool	Tham số	Bám dữ liệu	Kiểm chế	Kết thúc	Ý định ghi	Khối hiển thị
86%	97%	67%	100%	90%	100%	100%

Các điểm yếu được bảng điểm nêu ra: một kịch bản luôn ưu tiên tool tóm lược thay cho tool thống kê được kỳ vọng; một kịch bản kiểm chế gọi sai tool đọc; một kịch bản nghiên cứu web mở chạy 10-16 vòng tool trước khi trả lời; một tra cứu chi tiết kết thúc muộn. Bám dữ liệu là khía cạnh thấp nhất vì bộ chấm yêu cầu các giá trị cụ thể từ dữ liệu giả lập phải xuất hiện nguyên văn trong câu trả lời.

Bộ đo quy mô, `gpt-oss-120b` : một lượt chạy ban đầu với ít mẫu (k = 3) cho thấy việc chọn tool vẫn ổn định khi số tool được cung cấp tăng từ 4 lên 60; một lượt chạy sau đó với 60 tool (k = 8, ngày 2026-08-03) đạt trung bình 67%, nên khả năng mở rộng vẫn đang được đo lường. Với k = 16, ba phép thử rủi ro cao nhất (tra cứu chi tiết ở chế độ giọng nói, và hai biến thể có cuộc hội thoại trước đó gây nhầm lẫn) đều đạt 16 trên 16, khi chạy không có lưới an toàn đi sâu đến chi tiết.

Cách đọc các con số này

Đây là tỷ lệ đạt của bộ chấm trên các kịch bản có phản hồi tool giả lập, không phải độ chính xác đầu-cuối trên dữ liệu khách hàng. Payload thực lớn hơn và nhiều hơn dữ liệu giả lập, và bộ công cụ không chạy với connector thật. Bộ công cụ tồn tại để phát hiện hồi quy trong hành vi của prompt và vòng lặp khi mô hình, prompt hoặc guard thay đổi, và để những thay đổi đó đo lường được.

• **Tài sản kiểm thử khác**

363 tệp test bao phủ các thư viện, route và component giao diện (Vitest với Testing Library). Các test bất biến đáng chú ý: guard không thực thi mã, tiếp tục workflow bền vững, xác thực tick, độ tương phản giao diện và độ lệch từ điển i18n giữa ba ngôn ngữ. Các script A/B bổ sung so sánh nhà cung cấp và thiết lập suy luận, và một bộ công cụ phát lại chạy lại các cuộc hội thoại soạn workflow nhiều lượt.

A Phụ lục – Tham chiếu endpoint API

Chú giải xác thực: **Pub** công khai · **S** phiên · **M** phiên, thao tác thay đổi yêu cầu member trở lên · **R** owner hoặc admin · **T** bearer access token · **Tick** secret dùng chung. Mọi đường dẫn nằm dưới `/api`.

Phương thức	Đường dẫn	Mục đích	Xác thực
Xác thực và người dùng			
GET, POST	<code>/auth/[...nextauth]</code>	Handler của Auth.js	Pub
POST	<code>/register</code>	Đăng ký (mở / theo lời mời / đóng; 10 lần mỗi giờ mỗi IP)	Pub
GET	<code>/users</code>	Danh sách người dùng (chỉ chiếu các cột an toàn)	R
PATCH	<code>/users/[id]</code>	Đổi vai trò (owner; chặn mất owner cuối cùng) hoặc vô hiệu hoá (transaction thu hồi quyền)	R
PATCH	<code>/users/me</code>	Sửa hồ sơ của chính mình	S
GET, POST	<code>/access-tokens</code>	Liệt kê token của mình; tạo token (admin có thể tạo cho người dùng khác)	M

Phương thức	Đường dẫn	Mục đích	Xác thực
DELETE	/access-tokens/[id]	Thu hồi	S
Chat và trợ lý			
POST	/chat	Lướt chat dạng stream, hoặc xác nhận / từ chối một thao tác ghi đang chờ	M
GET	/chat/info	Mô hình mặc định và danh sách mô hình khả dụng	S
GET	/chat/tools	Danh mục tool của người gọi (nội bộ, connector, MCP)	S
GET, POST	/conversations	Liệt kê (tìm theo tiêu đề hoặc nội dung) / tạo	M
GET, PATCH, DELETE	/conversations/[id]	Tin nhắn kèm bảng dụng lại / đổi tên / xóa	M
GET, POST	/workspaces	Liệt kê / tạo workspace chat	M
PATCH, DELETE	/workspaces/[id]	Sửa / xóa (hội thoại trở thành không thuộc nhóm nào)	M
GET, POST	/custom-agents	Liệt kê / tạo preset	M
PATCH, DELETE	/custom-agents/[id]	Sửa / xóa	M
GET	/ollama/models	Danh sách mô hình cục bộ	S
POST	/fetch-url	Tải URL phía server (blocklist tên máy)	S
GET, POST	/ocr	Tình trạng sẵn sàng của OCR / nhận dạng (vi, en, zh)	S
GET, POST	/pdf	Trích xuất văn bản PDF (lớp văn bản, sau đó OCR, sau đó thị giác)	S
GET, POST	/docx	Trích xuất văn bản DOCX	S
POST	/tts	Chuyển văn bản thành giọng nói (WAV)	S
POST	/tts/stream	Chuyển văn bản thành giọng nói dạng stream (PCM)	S
GET	/geocode , /reverse , /route , /nearby	Geocoding, reverse geocoding, chỉ đường, địa điểm lân cận (có cache, có điều tiết tần suất)	S
GET	/weather	Thời tiết hiện tại	S
Connector và MCP			
GET	/connectors	Danh sách connector với thông tin xác thực đã che	S
GET, POST	/connectors/[id]/[action]	Kết nối / ngắt kết nối / kiểm tra / bắt đầu OAuth và callback	M
GET, POST, PATCH, DELETE	/connectors/mcp	MCP server theo từng người dùng (không bao giờ trả lại token)	M
POST	/connectors/mcp/test	Thăm dò server trước khi lưu	S
POST	/mcp	LAAM đóng vai MCP server (tool chỉ đọc)	T
Giám sát			
POST	/ingest	Collector đẩy dữ liệu (giới hạn body dạng stream 5 MB)	T
POST	/sync	Quét transcript cục bộ vào cơ sở dữ liệu	M
GET	/events	SSE: phiên, bước workflow, thông báo theo từng người dùng	S
GET	/stats	Số liệu tổng hợp cho dashboard	S
GET	/monitoring	Các lượt chạy được giám sát thống nhất (agent, chat, workflow)	S
GET	/agents/[id]/timeline	Nhật ký tin nhắn và tool của phiên	S
GET, POST	/machines	Liệt kê / tạo máy và cấp token cho collector	S / R
DELETE	/machines/[id]	Thu hồi token của collector	R
GET	/host/metrics	CPU, RAM, GPU của host (suy giảm êm)	S
GET	/config	Cấu hình runtime cho client	S
GET	/search	Tìm kiếm toàn cục	S
Workflow			
GET, POST	/workflows	Liệt kê (phân trang, kèm tổng số) / tạo	M
GET, PATCH, DELETE	/workflows/[id]	Đọc / cập nhật / xóa (xóa lan truyền lịch và lượt chạy)	M
POST	/workflows/[id]/run	Chạy ngay; chạy thử; graph nháp (chỉ chạy thử)	M
POST	/workflows/[id]/clone	Nhân bản workflow của mình hoặc template	M
POST	/workflows/[id]/save-as-template	Lưu graph thành template	M
GET, POST	/workflows/templates	Danh mục template / khởi tạo	M
POST	/workflows/templates/[id]/instantiate	Khởi tạo từ một template	M

Phương thức	Đường dẫn	Mục đích	Xác thực
GET	/workflows/runs	Các lượt chạy của người gọi (lọc theo workflow, trạng thái, mới nhất)	S
GET, PATCH	/workflows/runs/[id]	Chi tiết lượt chạy kèm các bước / huỷ	M
GET, POST	/workflows/schedules	Liệt kê / tạo lịch (kiểm tra cron hợp lệ)	M
PATCH, DELETE	/workflows/schedules/[id]	Bật, sửa, xoá	M
POST	/workflows/tick	Nhận các lịch đến hạn, thực thi các lượt chạy đang chờ và có thể tiếp tục	Tick
POST	/workflows/generate	Sinh bằng AI một lần (thăm dò, lập kế hoạch, kiểm chúng)	S
POST	/workflows/generate/chat	Một lượt trong phiên soạn workflow nhiều lượt bằng AI	S
POST	/workflows/generate/compile	Biên dịch vết đã quan sát thành graph (đường cũ)	S
POST	/workflows/generate/append-write	Nối thêm một bước ghi đã được con người xác nhận	S
POST	/workflows/review	Rà soát graph hiện tại bằng ngôn ngữ thông thường	S
Thông báo			
GET	/notifications	Thông báo của người gọi và số chưa đọc	S
PATCH	/notifications/[id]	Đánh dấu một thông báo đã đọc	S
POST	/notifications/read-all	Đánh dấu tất cả đã đọc	S